

AI-Native Practice Operations Platform

Principal AI Architecture — multi-agent orchestration, feature platform, custom training pipeline, trust and cost governance, and the core technology stack of a human-governed GenAI operating system for behavioural-health practices

Jyotirmoy Barman

Contents

- 1 Problem Statement 6**
 - 1.1 The Operational Gap Around the Clinical Session 6
 - 1.2 Why This Is an AI Problem, Not a Workflow Problem 6
 - 1.3 Problem Definition 6
 - 1.4 Constraints That Shape Every Answer 7
- 2 Project Overview 8**
 - 2.1 What Was Built 8
 - 2.2 Scope 8
 - 2.3 Platform Shape 8
 - 2.4 Programme Facts 9
- 3 Product Capability Baseline 9**
- 4 Real-Time and Operational Constraints 11**
 - 4.1 Quality Attribute Scenarios 11
 - 4.2 Planning Load Model 11
 - 4.3 Latency Budget for the First Reply 12
 - 4.4 Operational Constraints 13
- 5 Solutioning Approach 14**
 - 5.1 Design Principles 14
 - 5.2 Method 14
 - 5.3 Multi-Persona Control Plane 14
 - 5.4 Provider-Independent Orchestration 15
 - 5.5 Build-Versus-Buy Stance 15
- 6 Reference Architecture 16**
 - 6.1 Control Flow of a Single Inquiry 17
 - 6.2 Tenancy and Isolation Model 17
- 7 Multi-Agent Orchestration Fabric 18**
 - 7.1 Design Position 18
 - 7.2 Agent Inventory 19
 - 7.3 Orchestration Topology 21
 - 7.4 Graph State Contract 21
 - 7.5 Budgets, Loops and Termination 22

7.6 Human-in-the-Loop as a First-Class Primitive	22
7.7 Autonomy Policy	22
7.8 Tool Fabric: MCP Inside, A2A at the Edge	23
7.9 Failure Semantics	23
7.10 Orchestration Evaluation	24
8 Feature Platform	25
8.1 Why the Platform Needs a Feature Store	25
8.2 Entity Model	26
8.3 Feature Views	26
8.4 Streaming Compute	27
8.5 Online and Offline Stores	28
8.6 Point-in-Time Correctness	28
8.7 Request-Time Feature Computation	28
8.8 Governance and PHI Classification	28
8.9 Monitoring	29
9 Model Portfolio	30
9.1 Matching Ranker in Depth	30
9.2 Practice Voice in Depth	31
9.3 Safety Sentinel in Depth	32
10 Custom Training Pipeline	33
10.1 Principles	33
10.2 Pipeline Stages	34
10.3 Training Recipes	35
10.4 Tenant Data Governance in Training	35
10.5 Release Gates	36
10.6 Retraining Triggers	36
11 AI Control Plane	37
11.1 Model Gateway	37
11.2 Model Router	37
11.3 Context Compiler	38
12 Knowledge and Retrieval	39
12.1 Knowledge Domains	39
12.2 Retrieval Pipeline	39
12.3 Vector Store Topology	39

12.4 Embedding Pipeline	40
12.5 Practice Knowledge Graph and GraphRAG	40
13 Speech and Multilingual Intelligence	41
13.1 Voice Pipeline	41
13.2 Multilingual Pipeline	41
14 LLMOps, Observability and AI SRE	42
14.1 Artifact Inventory	42
14.2 Tracing Model	42
14.3 AI SLOs	42
14.4 Incident Classes	42
15 Resilience, Disaster Recovery and Continuity	44
15.1 RTO and RPO by Capability	44
15.2 Hybrid DR and Failover Topology	44
15.3 Continuity Lifecycle Orchestration	45
15.4 Circuit Breakers and Graceful Degradation	45
15.5 Resilience KPIs and Simulation-Based Validation	45
16 Core Tech Stack	47
16.1 Selection Method	47
16.2 Competency-to-Platform Mapping	47
16.3 Stack by Layer: Decisions in Depth	53
16.4 Multi-Cloud Equivalence	58
16.5 Deliberately Not Selected	59
17 Data Architecture and Interoperability	60
17.1 Operational Stores	60
17.2 Canonical AI Decision Record	60
17.3 Healthcare Interoperability	60
18 Canonical APIs	61
19 Trade-Offs and Decisions I Owned	62
19.1 The Decision Register	62
19.2 The Three Hardest Calls	63
20 Trust, Safety and Cost Governance	64
20.1 Governance Model	64
20.2 Threat Model	64

20.3 Red Teaming Programme	65
20.4 Adversarial Prompt Injection Defence in Depth	65
20.5 Data Exfiltration Path Analysis	66
20.6 Tenant-Level Network Policy Enforcement	66
20.7 AI-Agent-Led Compliance Audit	66
20.8 Clinical Boundary and HIPAA Safeguards	67
20.9 Cost Governance	67
20.10 Release Gates I Held	68
21 Authority	70
21.1 Decision Rights	70
21.2 Exercising the Veto	70
21.3 Authority Boundaries	71
22 Leadership	71
22.1 Team Structure	71
22.2 How I Led	71
22.3 Programme Leadership	72
23 Business Outcome	73
23.1 Outcomes Across Pilot Practices	73
23.2 How the Architecture Produced the Outcomes	73
23.3 Platform Economics	74
24 Architectural Impact	74
24.1 What the Architecture Changed	74
24.2 Reusable Patterns Established	74
24.3 Principal Assessment	75

1 Problem Statement

1.1 The Operational Gap Around the Clinical Session

Multi-clinician behavioural-health practices run their clinical record in an EHR, but everything that happens before the first session — and most of what keeps a client engaged after it — runs on inboxes, spreadsheets, sticky notes, voicemail and memory. A prospective client who writes at 21:42 on a Friday is typically answered on Monday. A coordinator juggling five tabs matches the client to whoever is free rather than whoever fits. Intake forms are chased over text and email. Referral sources are never told what happened to the people they sent. Monthly numbers come from a spreadsheet someone forgot to update.

The consequences are measurable and compounding. Every hour of response latency lowers the probability that a prospective client books at all, because people seeking therapy contact several practices in parallel and commit to the first credible answer. A poor first match increases early drop-out, which wastes the scarcest resource a practice has — clinician hours — and damages referral relationships. Leadership cannot see which channels, referrers, payers or clinicians actually convert, so hiring, credentialing and marketing decisions are made on instinct.

1.2 Why This Is an AI Problem, Not a Workflow Problem

Conventional practice-management and CRM tooling treats this as a routing problem: capture a lead, assign it, send a templated sequence. That fails on four properties of the domain.

Property	Why rules and templates break	What the platform must do instead
Unstructured, multilingual, multi-channel input	Inquiries arrive as free text, voicemail, call audio, referral PDFs and directory leads, in 30+ languages, with insurance, concerns and availability expressed indirectly	Interpret each inquiry into a typed, confidence-scored representation within seconds
Decisions under uncertainty with delayed outcomes	Whether a match was right is only known when the client attends, and again when they are still engaged at 90 days	Learn from outcomes that arrive days to months after the decision
Safety-critical tail	A small fraction of inquiries contain crisis, self-harm or abuse signals, often indirectly phrased, sometimes in a second language	Detect with guaranteed recall and remove automation from the path instantly
Practice-specific voice and policy	Each practice has its own tone, fees, payer panels, intake packets and escalation rules	Personalise generation per tenant without leaking one tenant's content into another's model

1.3 Problem Definition

The engineering problem is to build a multi-tenant, HIPAA-aligned, AI-native operating layer that sits beside the EHR and turns every inbound inquiry into a safe, fast, well-matched first session, with a human accountable at every consequential step, at a unit cost that holds under scale-based pricing.

Stated as measurable objectives:

Objective	Target
First on-brand reply, every day of the week	p50 <= 90s, p95 <= 150s; median first reply under two minutes
Voicemail transcribed, tagged and routed	p95 <= 60s from recording end
Language detection on every inbound message	< 1s, 30+ languages, thread continues in the detected language
Crisis-signal recall	>= 0.99 per supported language on an adjudicated golden set
Match quality	Right-fit matching measurably better than first-available, validated by attended and retained sessions, not by bookings alone
Staff time	Material reduction in coordinator and clinician administrative hours per week
Isolation	Zero cross-tenant exposure across data, caches, retrieval, features and model artifacts
Unit economics	Cost per inquiry lifecycle within plan at fleet load, attributable per tenant

1.4 Constraints That Shape Every Answer

- Protected health information is present in a large share of messages, recordings and documents; minimum-necessary use is a design invariant, not a policy aspiration.
- The platform is operational, not clinical. It must never diagnose, recommend treatment, interpret screener results for the client or counsel a person in crisis.
- The EHR remains the clinical system of record; the platform synchronises scheduling and handoff data bidirectionally and must reconcile, not overwrite.
- Tenants range from 5-clinician practices to 25+ clinician, multi-location networks; the architecture must serve both without a fork.
- Pricing is tied to practice scale rather than query volume, so the platform — not the customer — carries token-cost risk.

2 Project Overview

2.1 What Was Built

A greenfield, AI-native practice operations platform delivered from blank canvas to production over an 18-month programme. It unifies capture, response, matching, intake, scheduling, referral management, capacity planning and analytics for therapy practices, with a practice-voice Assistant that drafts triage replies, scheduling communications and routine follow-ups under human oversight, and that is structurally excluded from PHI it does not need.

2.2 Scope

Domain	Capability delivered
Capture	Unified inbox across web forms and chat, SMS, email, calls, voicemail, provider referrals and directory leads (Psychology Today, Zencare, GoodTherapy); auto-capture from missed calls; cross-channel entity resolution
Understand	Automatic tagging of insurance, urgency, presenting concern, modality, geography and preferred clinician traits; call and voicemail transcription and summarisation
Respond	Practice-voice replies and branded follow-up sequences; human takeover pauses automation instantly; 30+ language detection with full-thread localisation
Match	Eligibility on license, panel, modality, age band, language and caseload; learned ranking on fit, availability and outcomes; calibrated, overridable match scores; waitlist routing
Intake	Forms, consent, screeners (PHQ-9, GAD-7), pre-fill, SMS and email reminders, EHR handoff to SimplePractice and TherapyNotes
Schedule	Live availability from Google and Microsoft calendars and the EHR; transactional holds; confirmations and reschedules
Refer	Referral attribution to source, closed-loop status updates to referring providers, yield and revenue by referrer
Plan	Live caseload and utilisation, demand by specialty, waitlist analytics, hiring signals, multi-location forecasting
Measure	Source-to-revenue funnel by source, clinician and insurance; attribution to keyword; grounded weekly digest
Integrations	SimplePractice, TherapyNotes, Google Calendar, Outlook, Gmail, Twilio, HubSpot, Stripe, Slack, directories

2.3 Platform Shape

The platform is organised as a **5-tier distributed system** with two cross-cutting planes. Tiers own request flow; planes own control and governance.

Tier / plane	Owns	Non-negotiable property
T1 Channel and edge	Web, SMS, email, telephony, voicemail, directories, referrals; WAF, rate limits, consent, edge language ID	Idempotent ingestion; consent and opt-out enforced before any outbound byte
T2 Experience and API	Staff workspace, clinician and leadership views, GraphQL BFF, REST and gRPC service APIs (FastAPI)	Tenant claim in every token; no business logic in the BFF
T3 Orchestration	Durable lifecycle workflows, supervisor and specialist agents, human interrupts	Bounded autonomy; every side effect a typed, authorised, idempotent tool call
T4 Intelligence	Model gateway and router, inference serving, retrieval, speech, feature serving	Single egress for every model call; policy evaluated per call

Tier / plane	Owns	Non-negotiable property
T5 Data and knowledge	OLTP, event log, lakehouse, feature store, vector, lexical and graph stores	Tenant isolation by construction; point-in-time correctness for every training row
Control plane (cross-cutting)	Policy engine, autonomy policy, persona profiles, kill switches, configuration	Declarative, versioned, evaluated at runtime
Governance plane (cross-cutting)	Model factory, evaluation, red teaming, AI inventory, audit ledger, cost ledger	No artifact reaches production without lineage, an eval scorecard and a rollback path

2.4 Programme Facts

Attribute	Value
Delivery model	Greenfield, blank canvas to production; phased releases behind tenant-level flags
Duration	18 months to full platform capability; first production tenants at month 7
Team	28 engineers at peak across AI, agent engineering, ML platform, data platform, SRE, DevOps and QA, plus clinical safety and product
My role	Sole Principal AI Architect, accountable for platform architecture across the full lifecycle: control and data plane design, technology selection, trust and safety release gating, cost governance and resilience
Clouds	AWS primary; Azure as provider-independent secondary for inference failover and disaster recovery
Regulatory posture	HIPAA-aligned, BAAs across every PHI-bearing processor, encryption in transit and at rest, role-based access, clinical-grade audit logging

3 Product Capability Baseline

The following product behaviours are the requirements baseline. The rest of this document is the engineering response.

Domain	Required behaviour	Architectural implication
Capture	Unified inbox for web, calls, voicemail, email, referrals and directory leads; auto-capture from missed calls	Channel adapters with at-least-once delivery and a canonical inquiry envelope; cross-channel dedup is an entity-resolution problem
Tagging	Insurance, urgency, presenting concern, preferred clinician traits, modality, geography	Multi-label extraction with schema-constrained output; urgency is a safety-relevant signal, not a sort key
Voice	Calls transcribed and summarised; voicemail routed within 60 seconds	Streaming ASR, diarisation, summarisation and routing inside a 60s budget
Response	On-brand replies in the practice's voice in under two minutes; sequences pause when a human takes over	End-to-end latency SLO with the safety gate in-path; per-tenant voice model; takeover preemption semantics
Language	30+ languages detected in under one second; thread continues in detected language with consistent voice	Edge language ID; translation with terminology locks and back-translation QA; per-language eval sets
Matching	Specialty, identity, modality, insurance panel, live availability, fit, payer mix; overridable scores; waitlist	Hard-constraint eligibility, learned ranker, calibrated scores, override capture as training signal
Intake	Forms, consent, screeners, pre-fill, reminders, EHR handoff	Deterministic instrument scoring; FHIR-shaped internal model; reconciled EHR sync

Domain	Required behaviour	Architectural implication
Measure	Response time, match rate, intake completion, show rate by source, clinician and insurance; keyword attribution; weekly digest	Event-sourced funnel, attribution model, grounded narrative generation
Capacity	Live caseload, demand by specialty, waitlist, hiring recommendations, multi-location forecasting	Demand forecasting and capacity features in the feature platform
Trust	BAAs, encryption, role-based access, audit logs, Assistant excluded from unnecessary PHI	PHI minimisation as a compiler pass; attribute-based policy per model call

Jyotirmoy Bardhan

4 Real-Time and Operational Constraints

4.1 Quality Attribute Scenarios

Architecture is decided by its hardest scenarios, not its average ones. Every component in this document is traced against these.

ID	Scenario	Target	Design consequence
QA-1	Web inquiry arrives Sunday 21:42 in Spanish mentioning anxiety and BCBS	First on-brand reply p50 <= 90s, p95 <= 150s	Asynchronous pipeline with a pre-approved acknowledgement fast path and an enriched second message
QA-2	Inquiry contains self-harm ideation phrased indirectly in Spanish	Crisis path with recall >= 0.99; no generative reply; human paged <= 60s	Safety sentinel first in the graph, multilingual, ensemble-based and fail-closed
QA-3	94-second voicemail with heavy background noise	Transcribed, tagged, routed <= 60s	Chunked streaming ASR with early routing on partial transcript and later reconciliation
QA-4	Clinician blocks Thursday evening in Google Calendar	Matching stops offering those slots within 30s	Calendar webhooks to stream processor to online feature store; slot reservation is transactional
QA-5	Primary LLM provider degrades to 40% error rate	No reply-latency breach beyond 5 minutes; no unsafe output	Circuit breakers, cross-provider routing to Azure AI Foundry, template fallback, human queue
QA-6	Prompt injection in a referral PDF instructs the agent to email records externally	Zero exfiltration; attempt logged and alerted	Untrusted-content isolation, intent-scoped tool allow-lists, recipient egress policy, mesh egress deny
QA-7	Tenant offboards and requests deletion	All data, embeddings, features and adapters removed; shared models provably unaffected	Tenant-keyed lineage from raw record to training row to adapter; crypto-shredding
QA-8	New ranker version ships	No regression in attended-and-retained rate; no subgroup disparity regression	Offline counterfactual eval, shadow, interleaving, subgroup guardrails
QA-9	Full loss of the primary AWS region	RTO <= 60 min for the core lifecycle, RPO <= 5 min for operational state; reply path degraded but live within 15 min	Warm-standby region, Aurora Global Database, MSK replication, cross-cloud inference failover

4.2 Planning Load Model

The workload is sized from observed practice volumes — a single mid-sized practice processes roughly 400 inquiries a month, and six-location networks run 50+ clinicians — extrapolated to fleet scale. The numbers are deliberately conservative on the high side.

Dimension	Planning assumption	Derived load
Tenants (practices)	2,500 active	Long-tail distribution; top 5% of tenants generate ~30% of volume
Inquiries	~300 per tenant per month average	~750k inquiries/month; ~0.3/s mean; 8-12/s Monday-morning peak with burst factor 30x
Conversation events	~18 inbound and outbound messages per inquiry lifecycle	~13.5M messages/month

Dimension	Planning assumption	Derived load
Model invocations	9-14 per inquiry across classify, extract, safety, draft, translate, verify, summarise	~9M calls/month; 70% served by small or distilled models
Tokens	~38k input and ~3k output per inquiry lifecycle before caching	~28B input tokens/month; prefix caching removes 45-60% of billable input
Voice	~25% of inquiries include a call or voicemail; mean 2.1 minutes	~400k audio minutes/month
Calendar and EHR sync	~40k clinicians x ~12 events/day	~15M sync events/month into the stream processor

The load is not throughput-hard; it is latency-, correctness- and isolation-hard. The platform is optimised for tail latency on a low-QPS, bursty, multi-tenant workload with an expensive, variable-latency dependency in the critical path. That single observation drives the decision to build on durable workflows rather than synchronous request chains, and to put a hard-budgeted small-model tier ahead of frontier models.

4.3 Latency Budget for the First Reply

The sub-two-minute reply is only credible with an explicit budget. The first reply is split into a guaranteed acknowledgement path and an enriched follow-on path.

Stage	Budget p95	Component	Degraded mode
Channel ingest, dedup, envelope	1.5s	Channel adapter, idempotency store	Queue and retry; never drop
Language identification	0.2s	Edge language-ID model	Default to practice primary language; flag for review
Safety sentinel (tier 1 + tier 2)	1.8s	Fine-tuned multilingual classifier + rules on Triton; LLM judge only on ambiguous band	Fail-closed to human queue with crisis resources template
Structured extraction	2.5s	Distilled multi-task encoder; LLM fallback on low confidence	Partial tags; human completes
Policy and consent check	0.1s	Policy decision point (cached)	Block outbound
Draft generation	6-12s	Tenant voice adapter on vLLM, or frontier model for complex threads	Pre-approved localised template
Verifier and output guardrails	2.0s	Schema, PII egress, claims checker, tone	Template fallback
Autonomy decision and dispatch	1.0s	Autonomy policy + channel send	Queue to staff approval
Total, auto-send tier	~27s p95 compute	Leaves > 90s headroom for queueing, provider variance and retries	—

The headroom is intentional. At p99, hosted LLM endpoints exhibit multi-second tails and occasional 30-60s stalls; the budget absorbs one full retry on an alternate provider without breaching the objective.

4.4 Operational Constraints

Constraint	Source	Architectural response
After-hours coverage with no staff online	94% same-day reply objective; inquiries peak evenings and weekends	Autonomy tiers that allow pre-approved intents to auto-send; crisis path pages an on-call human regardless of hour
Calendar and EHR APIs are rate-limited and lossy	Provider throttling, expired webhook subscriptions	Webhooks as latency optimisation over an authoritative poll; re-validate before every hold
Staff are the approval capacity	Small practices have one or two coordinators	Review queues prioritised by SLA risk; autonomy adapts to measured reviewer latency
Bursty Monday-morning load	Weekend backlog	Per-tenant fair queuing; KEDA-driven autoscaling on queue depth; GPU pools with warm minimums
Multi-location networks	6+ offices, 50+ clinicians	Location-aware routing; dedicated cells for the largest tenants
Regulatory evidence on demand	BAAs, audits, customer security reviews	Continuous, agent-assisted evidence collection against a controls catalogue
Token-cost exposure	Scale-based pricing	Cost per inquiry as an SLO with per-tenant attribution and anomaly alerting

5 Solutioning Approach

5.1 Design Principles

Three rules govern the solution. First, a model never owns business state; it proposes, a deterministic service commits. Second, every probabilistic component has a measured quality floor, a cost ceiling and a degraded mode that is safer than its happy path. Third, nothing crosses the PHI boundary that the task does not require; the Assistant is excluded from identifiers by construction, not by instruction.

5.2 Method

I ran the solutioning in five deliberate steps, each producing an artifact that the next step depended on.

- 1. Decision inventory before component inventory.** I decomposed the product into the 31 decisions it makes — classify urgency, detect language, choose clinician candidates, place a hold, choose send-or-stage, choose follow-up timing, and so on — and classified each by reversibility, PHI exposure, latency budget and whether a human is accountable. Components were derived from decisions, not the reverse.
- 2. Deterministic-versus-probabilistic boundary.** Each decision was assigned to deterministic code, a learned model, a foundation model, or a human, with the rule that anything with a legal or financial commitment is deterministic or human. This produced the deterministic core and the agent charters.
- 3. Hard-scenario tracing.** The quality attribute scenarios were written first and every candidate architecture was walked through them. QA-2 (indirect crisis signal) and QA-6 (indirect prompt injection) eliminated three early candidate designs that placed a single LLM agent in control of both reading untrusted content and calling tools.
- 4. Spike-validated technology selection.** Every contested choice was settled by a time-boxed prototype with a pre-agreed metric: recall and latency benchmarks for the safety sentinel, p95 and cost per 1k drafts for managed versus self-hosted voice models, cutover time for cross-cloud inference failover.
- 5. Closed learning loop by design.** The decision record, the human review schema and the outcome events were designed together, so that every production decision becomes a future training row from day one.

5.3 Multi-Persona Control Plane

The platform serves six personas with different trust levels, tools and data entitlements. Rather than building six assistants, the control plane defines persona profiles that parameterise the same agent fabric.

Persona	Surface	Data classes permitted	Tools	Autonomy ceiling	Model route
Prospective client	SMS, email, web chat	Own operational data only	None directly; agents act on their behalf	A3 for approved intents	Tenant voice adapter; frontier for complex threads
Intake coordinator	Staff workspace	Operational and PHI-derived for assigned inquiries	Review, edit, send, match, schedule	Human is the authority	Staff copilot routes
Clinician	Clinician view	Own caseload and profile	Profile and directory drafts, availability	A1	Mid-tier LLM

Persona	Surface	Data classes permitted	Tools	Autonomy ceiling	Model route
Practice leadership	Dashboards, digest	Aggregates; no message bodies	Metrics semantic layer, forecasts	A1	Mid-tier LLM with numeric grounding
Billing and credentialing specialist	RCM workspace	Claims and credentialing data	Correspondence drafts, status summaries	A1; never submits	Frontier LLM with payer retrieval
Platform operator	Operations console	Metadata, traces with PHI redacted	Kill switches, replays, config	Break-glass only	—

A persona profile is a signed, versioned policy bundle evaluated by the policy engine on every model and tool call: which tools, which data classes, which model endpoints, which tone and which autonomy ceiling apply. Adding a persona is a configuration change reviewed like code, not a new service.

5.4 Provider-Independent Orchestration

Orchestration never binds to a model vendor. Agents call the model gateway by capability class (`draft.practice_voice`, `classify.safety`, `verify.policy`) rather than by model name, the gateway resolves capability to endpoint by policy, and tools are exposed over MCP rather than vendor-specific function formats. Swapping the primary provider for a capability class is a router policy change validated by trace replay, not a code change — which is what made the cross-cloud inference failover in QA-5 and QA-9 achievable.

5.5 Build-Versus-Buy Stance

Capability	Decision	Reasoning
Foundation models	Buy (managed, multi-provider)	Commodity capability; differentiation lies elsewhere
Voice adapters, encoders, ranker, sentinel	Build	Proprietary data and outcome loops are the moat
Durable workflows	Buy the runtime (Temporal), build the workflows	Durable execution is hard to build and easy to buy
Agent graph runtime	Adopt open source (LangGraph)	Explicit control over topology and state
Model gateway	Build on an open core	Policy, PHI-class routing and budgets are platform-specific
Guardrails	Compose (Bedrock Guardrails + NeMo rails + custom validators)	No single product covers PHI egress, clinical boundary and injection defence
Red teaming	Adopt (PyRIT, Giskard, garak) plus in-house attack corpora	Tooling is commodity; attack corpora are domain-specific
Feature store	Adopt open definitions (Feast) on managed stores	Portability of definitions matters more than managed UX
Observability	Adopt standards (OpenTelemetry)	Vendor-neutral traces across two clouds

6 Reference Architecture

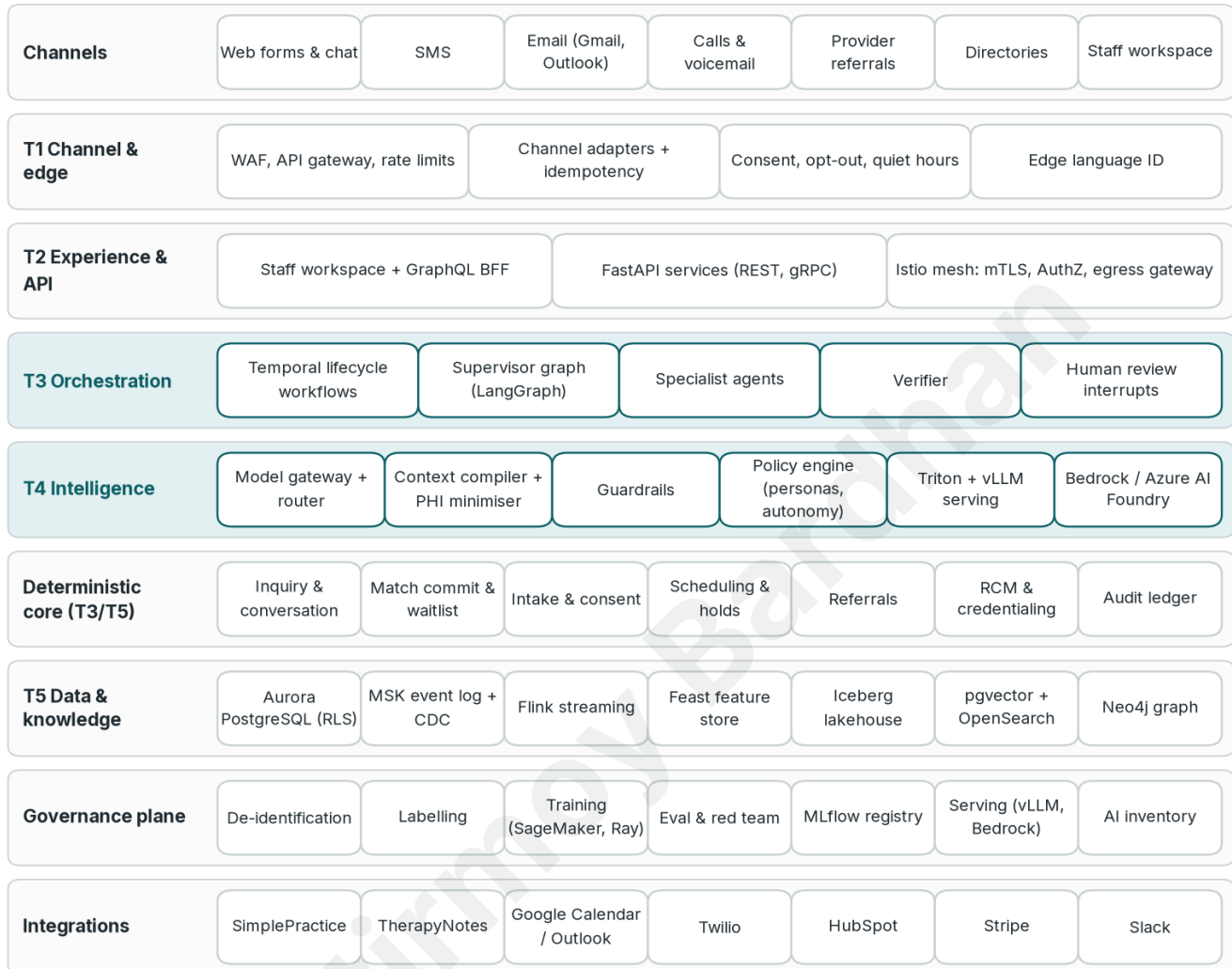


Figure 1. Reference architecture — planes and principal components

The architecture is a cell-based, event-sourced system. A cell is a self-contained deployment of the operational core, orchestration workers and data stores serving a shard of tenants; the AI control plane and model factory are regional shared services with per-tenant policy. Cells bound blast radius — a runaway agent loop or a poisoned index affects one cell, not the fleet — and give large multi-location tenants a path to dedicated cells without a fork of the codebase.

6.1 Control Flow of a Single Inquiry

1. A channel adapter normalises the payload into an `InquiryEnvelope`, computes a content hash and channel-native idempotency key, and appends `inquiry.received` to the tenant-partitioned event log.
2. The orchestration fabric starts, or signals, a durable `InquiryLifecycle` workflow keyed by `tenant_id:inquiry_id`. The workflow is the single writer of lifecycle state.
3. The workflow's first activity invokes the Assistant supervisor graph, which fans out in parallel to the safety sentinel, the language agent and the extraction agent. The safety verdict is a join barrier: no downstream node executes until it resolves.
4. The context compiler assembles a minimum-necessary context for each subsequent node from the operational store, the online feature store and the tenant knowledge index, applying PHI minimisation and a token budget.
5. Specialist agents propose actions — a draft, a ranked clinician list, a slot hold, a form pre-fill. Each proposal is a typed object, never free text for a machine to parse.
6. The autonomy policy maps each proposal, its risk class, confidence and tenant configuration to an execution mode: auto-execute, stage for approval, or escalate.
7. Deterministic services validate and commit. Commits emit events; events update features, analytics and the audit ledger; outcomes arriving days later (booked, showed, retained) are joined back to the originating decision as labels.

6.2 Tenancy and Isolation Model

Layer	Isolation mechanism	Why this and not less
Identity	Tenant-scoped OIDC realms; tenant claim is signed and propagated in every token and span	Tenant context cannot be inferred from request data, only from identity
OLTP	Shared-schema PostgreSQL with row-level security keyed on <code>tenant_id</code> ; large tenants on dedicated cell databases	RLS enforces isolation at the engine, not the ORM; dedicated cells for noisy neighbours
Encryption	Per-tenant data keys under envelope encryption with a KMS customer master key per cell; crypto-shredding on offboarding	Deletion that survives backups and replicas
Event log	Partition key begins with <code>tenant_id</code> ; consumers enforce tenant filter; per-tenant quotas	Ordering per tenant, fairness across tenants
Vector store	Tenant-partitioned pgvector tables with partition-pruned ANN; dedicated OpenSearch k-NN indexes for top tenants	Filtered ANN without per-tenant index explosion for the long tail
Feature store	Composite entity keys <code>tenant_id#entity_id</code> ; feature views declare tenant scope or global scope explicitly	Global features (payer behaviour) cannot silently leak tenant specifics
Models	Global base models trained only on de-identified, contractually permitted data; tenant adapters confined to tenant scope	Voice adaptation without cross-tenant memorisation
Network	Namespace per cell with default-deny Kubernetes NetworkPolicies, Istio mTLS and AuthorizationPolicy, egress only via an Istio egress gateway to allow-listed destinations	An agent that is tricked into calling an unknown host cannot reach it
Caches	Cache keys include tenant, policy version and PHI class; semantic cache disabled for PHI-bearing prompts	The most common GenAI data leak is a shared cache

7 Multi-Agent Orchestration Fabric

7.1 Design Position

The platform's agent layer is a **supervised hierarchical multi-agent system executing inside durable workflows**. It is deliberately not an open-ended group chat of agents negotiating in natural language. Free-form multi-agent conversation is attractive in demos and pathological in production: unbounded token spend, non-reproducible traces, emergent loops, and no clear owner for a side effect. In a regulated operational domain every agent must have a narrow charter, a typed input and output contract, an explicit tool allow-list and a budget, and the orchestration topology must be declared, versioned and testable as code.

The fabric has two runtimes with a strict division of labour:

Runtime	Responsibility	Time horizon	Technology
Durable workflow runtime	Business lifecycle: inquiry, follow-up cadences, intake, scheduling holds, referral loops, sagas and compensations, timers, human waits	Seconds to 90 days	Temporal ¹
Agent graph runtime	Reasoning inside one workflow step: supervisor routing, specialist execution, parallel fan-out, reflection, checkpointed state, human interrupts	Milliseconds to minutes	LangGraph ²

The rule is simple and enforced in code review: **anything that must survive a deploy, wait for a human, wait for time, or compensate on failure is a workflow concern; anything that reasons over context to produce a proposal is a graph concern**. Temporal owns the durable, replayable history of what happened; LangGraph owns the checkpointed, inspectable state of how the proposal was reached. Graph invocations are executed as Temporal activities with heartbeats, deterministic retry policy and idempotency keys, so an agent step that times out is retried or re-routed without duplicating a message or a slot hold.

1. Temporal — Workflow Execution — <https://docs.temporal.io/workflow-execution>

2. LangGraph overview — LangChain docs — <https://docs.langchain.com/oss/python/langgraph/overview>

Durable workflow: InquiryLifecycle (Temporal) — timers, signals, sagas, human waits

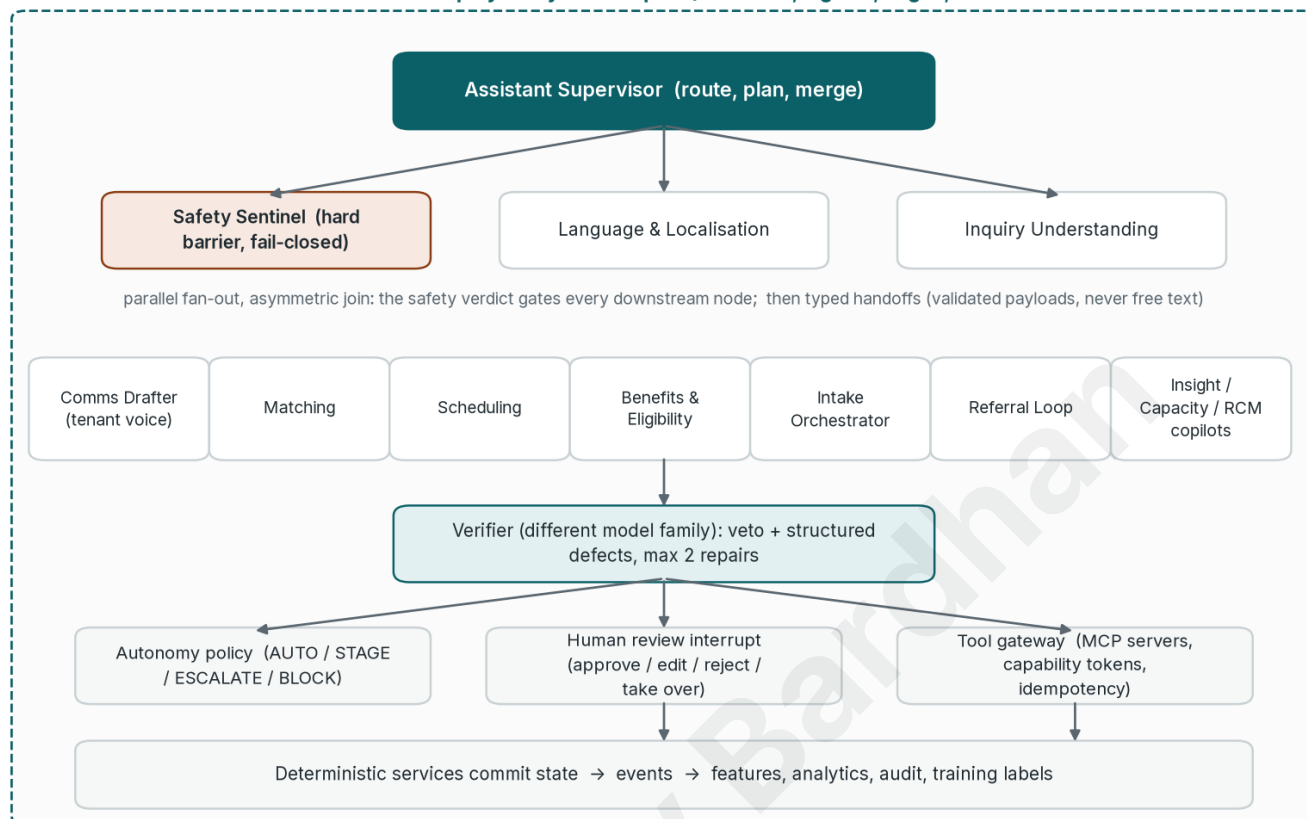


Figure 2. Multi-agent orchestration fabric — supervised hierarchy inside a durable workflow

7.2 Agent Inventory

Each agent is a versioned artifact comprising a system prompt, a structured-output schema, a tool manifest, a model routing policy, an eval suite and an autonomy ceiling. Agents are stateless between invocations; all state lives in the typed graph state or in the operational core.

Agent	Charter	Primary model tier	Tools (allow-list)	Autonomy ceiling
Assistant Supervisor	Classifies intent and phase, plans the node sequence, routes to specialists, merges proposals, requests human review	Mid-tier LLM with constrained decoding	None with side effects; handoff, request_review only	A1 — cannot act, only route
Safety Sentinel	Detects crisis, self-harm, harm to others, abuse disclosure, medical emergency, minors at risk; enforces the clinical boundary	Fine-tuned multilingual classifier ensemble; LLM judge for the ambiguous band only	escalate_crisis, page_on_call, lock_automation	Fail-closed; overrides every other agent
Language and Localisation	Language ID, locale, register and script; translation with terminology locks; back-translation QA	Edge language-ID; MT or LLM translation with glossary	get_glossary, get_locale_policy	A3 for templated messages; A1 for free text in low-resource languages

Agent	Charter	Primary model tier	Tools (allow-list)	Autonomy ceiling
Inquiry Understanding	Extracts presenting concerns, insurance, modality, availability windows, clinician trait preferences, urgency, referral source	Distilled multi-task encoder; LLM fallback when calibrated confidence < threshold	lookup_payer_alias, resolve_referrer	A0 — produces facts, not actions
Communications Drafter (Practice Voice)	Produces practice-voice drafts for triage replies, follow-ups, reminders and scheduling messages	Tenant LoRA adapter on open-weight model; frontier model for complex or sensitive threads	get_practice_policies, get_availability_summary, retrieve_kb	A3 on approved intents; A1 otherwise
Matching Agent	Invokes eligibility and ranking, explains the ranked list, surfaces trade-offs and waitlist options	Ranker service plus small LLM for explanation	match.eligible, match.rank, waitlist.propose	A2 — recommend; human or policy commits
Scheduling Agent	Proposes slots, places short-lived holds, handles reschedule intents, confirms appointments	Small LLM for intent parsing; deterministic slot engine	slots.search, slots.hold, slots.release, appointment.propose	A3 for holds; A2 for confirmation on first sessions
Benefits and Eligibility	Normalises payer and plan, checks panel participation, prepares eligibility verification, drafts client-facing insurance explanations	Small LLM with payer knowledge retrieval	payer.normalize, panel.check, eligibility.request	A2
Intake Orchestrator	Selects the intake packet, pre-fills from inquiry facts, tracks completion, chases missing items	Small LLM plus deterministic form engine	intake.packet_select, intake.prefill, intake.status	A3 for reminders
Referral Loop Agent	Parses referral documents, links referrer, drafts closed-loop status updates to referring providers	Document LLM with layout parsing	referral.link, referral.status_draft	A1 for provider communications
Directory Content Agent	Drafts SEO-focused clinician directory copy from approved clinician profiles	Frontier LLM	profile.read, directory.draft	A1 — clinician approves
Capacity and Growth Planner	Explains demand-by-specialty, waitlist and hiring signals from the forecasting models	Mid-tier LLM over forecast outputs	forecast.read, capacity.read	A1
Insight Narrator	Produces the weekly digest and dashboard narratives strictly from computed metrics	Mid-tier LLM with numeric grounding checks	metrics.query (read-only semantic layer)	A1; numbers must be tool-sourced
RCM and Credentialing Copilot	Assists human billing and credentialing staff with payer correspondence, denial categorisation and status summarisation	Frontier LLM with retrieval over payer rules	claims.read, credentialing.read, draft_letter	A1 — never submits
Verifier (Critic)	Independent check of any outbound draft: factual consistency with state, policy conformance, PHI egress, tone, promise-making	Different model family from the drafter	state.read, policy.read	Veto only

The Verifier is intentionally a different model family from the Drafter. Correlated errors between generator and judge are the silent failure of single-vendor agent stacks; model diversity is cheap insurance.

7.3 Orchestration Topology

The topology is a supervisor graph with three structural patterns, each used where it is the correct tool and nowhere else.

Parallel fan-out with a safety barrier. On every new inbound message the supervisor fans out to Safety, Language and Understanding concurrently. The join is asymmetric: Safety is a hard barrier and short-circuits the graph on a positive or indeterminate verdict; Language and Understanding results are merged into state when Safety clears. Parallelism removes roughly two seconds from the critical path without weakening the safety invariant.

Typed handoff. Specialists are invoked through a `handoff(target, reason, payload_schema)` primitive. A handoff is a state transition in the graph, not a message in a transcript. The supervisor cannot pass arbitrary text to a specialist; it passes a validated payload. This eliminates the class of failures where one agent's hallucination becomes another agent's instruction.

Generate-verify-repair. Every outbound artifact passes through the Verifier. A veto returns structured defects (`UNSUPPORTED_CLAIM`, `POLICY_CONFLICT`, `PHI_EGRESS`, `TONE_DRIFT`, `LANGUAGE_MISMATCH`) to the Drafter for at most two repair iterations. After the second failure the artifact degrades to an approved template or a human queue. Unbounded self-reflection loops are prohibited; empirically, quality gains after the second repair are marginal and cost grows linearly.

7.4 Graph State Contract

The graph state is a typed, versioned schema. Agents read and write only the fields their manifest declares; writes are reducer functions, so parallel branches merge deterministically.

```
AssistantState v7
tenant_id, inquiry_id, thread_id, workflow_run_id, policy_bundle_version
phase:          CAPTURE | RESPOND | MATCH | INTAKE | SCHEDULE | NURTURE | CLOSED
channel:         WEB | SMS | EMAIL | VOICE | VOICEMAIL | REFERRAL | DIRECTORY
language:        { code, script, confidence, source: EDGE_LID | USER_STATED }
safety:          { verdict: CLEAR | ELEVATED | CRISIS | INDETERMINATE,
                  signals[], model_versions[], latency_ms }      # reducer: max-severity wins
facts:           { concerns[{code, span_ref, conf}], payer{raw, normalized, plan_type, conf},
                  modality_pref, availability_windows[], trait_prefs[], age_band,
                  referral_source_ref }                          # reducer: confidence-weighted merge
context_refs:    [ { kind: KB_CHUNK | FEATURE_VECTOR | EVENT, id, version, phi_class } ]
proposals:       [ { id, kind: DRAFT | MATCH_LIST | SLOT_HOLD | FORM_PREFILL | ESCALATION,
                  payload_ref, risk_class, confidence, produced_by, verifier_status } ]
human:           { assigned_to, takeover: bool, pending_review_ids[] }
budget:          { tokens_remaining, steps_remaining, deadline_utc, cost_ceiling_usd }
trace:           { otel_trace_id, span_ids[] }
```

Two details matter. Free text from the client is never stored in `facts` directly; facts hold normalised codes and span references into the encrypted message store, which keeps downstream prompts small and keeps raw PHI out of state checkpoints. And `human.takeover` is checked before every side-effecting node — the product requirement that sequences pause when a human takes over is implemented as a workflow signal that flips this flag and cancels pending timers atomically.

7.5 Budgets, Loops and Termination

Every graph invocation carries a budget object enforced by the runtime, not by prompt instructions.

Guard	Default	Enforcement
Max graph steps per invocation	12	Runtime raises BudgetExceeded; workflow routes to human
Max repair iterations	2	Edge condition on Verifier node
Token ceiling per invocation	24k input / 2k output	Gateway rejects calls exceeding remaining budget
Wall-clock deadline	Derived from the phase SLO	Activity timeout; alternate route on expiry
Cost ceiling	Per-tenant, per-intent	Gateway pre-flight estimate; downgrade model tier before rejecting
Cycle detection	Same (node, state-hash) visited twice	Hard stop; trace flagged for review

7.6 Human-in-the-Loop as a First-Class Primitive

Human review is modelled as an interrupt in the graph and a signal-wait in the workflow, never as a UI convention. A review item carries the proposal, the minimum context needed to judge it, the verifier findings, the model and prompt versions and a structured decision form: approve, edit-and-approve, reject with reason code, take over. The decision is written back as an event and becomes a training label: approvals and rejections train the autonomy policy; edits become preference pairs for the voice models; overrides of match rankings become counterfactual signals for the ranker. The review UI is therefore part of the training pipeline, and its schema is owned jointly by product and ML.

7.7 Autonomy Policy

Autonomy is decided per proposal by a policy function, not by the agent that produced it.

```
mode = autonomy_policy(  
  intent_class,      # e.g. ACK_FIRST_CONTACT, FOLLOWUP_DAY2_INSURANCE, RESCHEDULE, FIRST_SESSION_CONFIRM  
  risk_class,        # derived from safety verdict, PHI class, commitment type, reversibility  
  calibrated_confidence, # from verifier and drafter, isotonic-calibrated per intent  
  tenant_autonomy_config, # practice-level opt-ins, business hours, reviewer availability  
  channel, language_tier, # low-resource languages cap autonomy at A1  
  recent_quality_signal # rolling edit-distance and rejection rate for this tenant x intent  
) -> AUTO_SEND | STAGE_FOR_APPROVAL | ESCALATE | BLOCK
```

The `recent_quality_signal` term is what makes autonomy earned rather than configured. If the rejection rate for an intent in a tenant rises above its control limit, that intent is automatically demoted to approval mode until quality recovers. This is a production control loop on autonomy, and it is the single most effective mechanism for keeping an agentic system safe as models, prompts and tenant behaviour drift.

7.8 Tool Fabric: MCP Inside, A2A at the Edge

All agent side effects go through a tool gateway that exposes internal capabilities as Model Context Protocol servers.³ MCP is used for what it is good at — discoverable, schema-described tools and resources with a uniform invocation contract — and the gateway adds what MCP does not provide on its own: per-call authorisation, capability tokens, rate limits, idempotency, data-class egress policy and audit.

Tool domain	MCP server	Side effect class	Authorisation
Inquiry and conversation	inquiry-mcp	Write (internal state)	Capability token bound to inquiry_id
Messaging dispatch	comms-mcp	External, irreversible	Only callable by workflow activity after autonomy decision; never directly by an LLM
Scheduling	schedule-mcp	External, reversible (holds), irreversible (confirm)	Holds auto-expire; confirmation requires autonomy=AUTO_SEND or human token
Matching	match-mcp	Read, compute	Read-only
EHR connectors	ehr-simplepractice-mcp, ehr-therapynotes-mcp	External, PHI-bearing	Server-side only; LLM never sees credentials; payloads constructed deterministically
Knowledge	kb-mcp	Read	Tenant-filtered retrieval
Metrics	metrics-mcp	Read	Semantic-layer queries only; no raw SQL from models

Capability tokens are short-lived, audience-restricted and scoped to a single entity and verb set — for example, `schedule:hold on clinician_id=c_482` for 10 minutes. An agent that has been prompt-injected cannot escalate beyond the capability it was minted, and it cannot address an entity outside the current inquiry.

The Agent2Agent protocol is used at the organisational boundary, not inside the platform.⁴ Where the platform interoperates with agents operated by partners — an outsourced billing service's claims agent, or a referring health system's care-coordination agent — A2A provides agent discovery via agent cards, task lifecycle and asynchronous artifacts without exposing internal tools. Internally, A2A would add a network hop and a negotiation protocol to what is better expressed as a typed function call; externally, it is the right abstraction because neither side should see the other's tools or prompts.

7.9 Failure Semantics

Failure	Detection	Response
Model timeout or 5xx	Gateway circuit breaker per provider and model	Route to equivalent model on alternate provider; if exhausted, template path
Schema-invalid output	Constrained decoding plus validator	One retry with error feedback; then fallback model; then template
Verifier veto twice	Graph edge	Human queue with defects attached
Tool call rejected by policy	Tool gateway	Recorded as a security event; graph re-plans without that tool or escalates
Duplicate side effect risk after retry	Idempotency key (workflow_run_id, activity_id, attempt-invariant hash)	Tool gateway returns the original result
Human takeover mid-graph	Workflow signal	Cancel scope; release holds; suppress pending sends

3. Model Context Protocol — Tools specification — <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>

4. Agent2Agent (A2A) Protocol specification — <https://github.com/a2aproject/A2A/blob/main/docs/specification.md>

Failure	Detection	Response
Safety indeterminate	Sentinel verdict	Treat as elevated; human within SLA; no generative outbound

7.10 Orchestration Evaluation

Agents are evaluated at three levels, because end-to-end metrics alone cannot localise regressions.

Level	What is measured	Method
Node	Extraction F1 per field, safety recall and precision, translation adequacy, draft policy conformance	Golden sets per node and language; deterministic graders plus calibrated LLM judges
Trajectory	Correct routing, tool-call correctness, unnecessary steps, budget use, handoff payload validity	Replay of recorded traces through a candidate graph; trajectory diff against reference
Outcome	Time to first reply, human edit distance, approval rate, booked-and-showed rate, complaint rate	Online, by tenant, intent and language, with guardrail metrics

Graph changes ship through trace replay: a candidate graph version is run against a stratified sample of last week's production traces with tools stubbed from recorded responses. Any divergence in tool-call sequence on safety-relevant traces blocks release.

8 Feature Platform

8.1 Why the Platform Needs a Feature Store

Three of the platform's headline behaviours are feature problems before they are model problems. Matching on "live availability" requires a fresh, consistent view of clinician capacity at decision time. Hiring recommendations "based on inquiry growth and team load" require time-series aggregates by specialty and location. And any claim of 2.4x right-fit matching requires that the features used to rank at decision time are exactly reproducible when the ranker is retrained — otherwise the offline evaluation is measuring a different system from the one in production.

The feature platform exists to guarantee four properties: point-in-time correctness for training, low-latency consistent reads for serving, one definition per feature shared by both paths, and governed lineage from raw event to feature to model to decision. It is built on Feast as the registry and serving abstraction, with a streaming compute tier and an Iceberg offline store, and SageMaker Feature Store as the managed alternative evaluated for cells where operational simplicity outweighs portability.^{5,6}

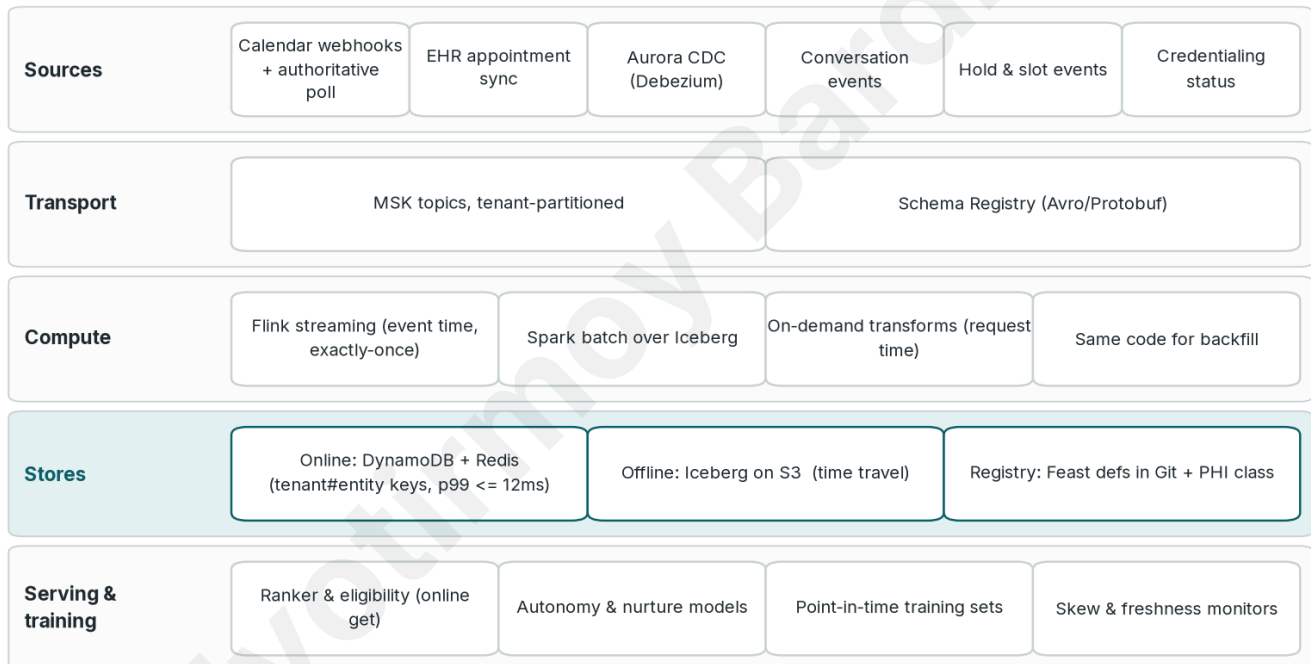


Figure 3. Feature platform — streaming and batch compute into online, offline and registry

5. Feast — Feature view concepts — <https://docs.feast.dev/getting-started/concepts/feature-view>

6. Amazon SageMaker Feature Store — <https://docs.aws.amazon.com/sagemaker/latest/dg/feature-store.html>

8.2 Entity Model

Entity	Key	Scope	Notes
Tenant	tenant_id	Tenant	Practice-level configuration and aggregate behaviour
Location	tenant_id#location_id	Tenant	Multi-location networks; utilisation and conversion by office
Clinician	tenant_id#clinician_id	Tenant	Capacity, specialty, modality, panel participation, performance
Inquiry	tenant_id#inquiry_id	Tenant	Request-time and short-lived features; the primary ranking context
Prospect	tenant_id#prospect_pid	Tenant, PHI	Pseudonymous identifier; features restricted to operational history, never clinical content
Referral source	tenant_id#referrer_id	Tenant	Yield, conversion, show rate, revenue attribution
Payer plan	payer_plan_id	Global	Normalised payer and plan; cross-tenant aggregates are k-anonymised and differentially noised
Slot	tenant_id#clinician_id#slot_start	Tenant	Materialised availability; highest-churn entity in the system
Clinician x payer	tenant_id#clinician_id#payer_plan_id	Tenant	Panel status, historical show and retention by payer

8.3 Feature Views

Feature views are grouped by freshness class. Freshness is a contract with an SLO and an alert, not a description.

Feature view	Representative features	Compute	Freshness SLO	Consumers
clinician_capacity_rt	open_slots_next_7d, open_evening_slots_7d, open_telehealth_slots_7d, next_available_ts, caseload_pct, hold_count_active	Streaming (calendar webhooks, EHR appointment CDC, hold events)	<= 30s event-to-online	Matching eligibility and ranking, scheduling, capacity views
clinician_profile	specialty_codes, modality_codes, languages, identity_attributes (self-declared, consented), age_bands_served, profile_embedding_ref	Batch on change	<= 5 min	Eligibility, ranker, explanation
clinician_performance_90d	match_accept_rate, consult_to_book_rate, first_session_show_rate, 90d_retention_rate, by-payer and by-concern breakdowns	Batch daily with Bayesian shrinkage to tenant prior	<= 26h	Ranker, capacity planner, analytics
clinician_payer_panel	panel_status, credentialing_stage, effective_date, historical_show_rate_by_payer	CDC from credentialing service	<= 2 min	Eligibility hard filter, benefits agent
inquiry_request	channel, local_hour, weekday, language, hours_since_received, concern_codes, payer_normalized, modality_pref, availability_window_overlap	On-demand at request time	Request time	Ranker, autonomy policy, drafter context

Feature view	Representative features	Compute	Freshness SLO	Consumers
inquiry_engagement_rt	replies_count, last_reply_latency, sentiment_trend, form_progress_pct, reminders_sent, human_touch_count	Streaming	<= 60s	Nurture cadence, no-show and drop-off models
referral_source_stats	inquiries_90d, schedule_rate, show_rate, revenue_90d, momentum (EWMA slope)	Batch daily	<= 26h	Attribution, referral agent, insight narrator
tenant_demand_ts	inquiries_by_specialty_week, waitlist_len, avg_wait_days, unmet_demand_by_payer	Batch hourly	<= 2h	Demand forecasting, hiring signal
payer_global	days_to_eligibility, denial_rate_band, out_of_network_share	Batch weekly, cross-tenant, privacy-protected	<= 8d	Benefits agent, forecasting

The `clinician_performance_90d` view uses empirical-Bayes shrinkage toward the tenant and specialty prior. Raw rates on small denominators are the single largest source of ranking instability in marketplace-style matching: a new clinician with two shows out of two should not outrank a veteran at 88% over 300 sessions. Shrinkage also prevents the ranker from learning a proxy for tenure, which would entrench incumbents and starve onboarding clinicians — the exact clinicians the capacity view labels "room to grow".

8.4 Streaming Compute

Real-time features are computed by Apache Flink jobs consuming the tenant-partitioned event log.⁷ Sources are calendar provider webhooks (Google, Microsoft Graph) and EHR appointment changes, normalised into `slot.*` and `appointment.*` events; hold events from the scheduling service; conversation events from the messaging service.

Concern	Design
Keying	<code>keyBy(tenant_id, clinician_id)</code> ; state is tenant-local, so rebalancing never mixes tenants in one key group
Event time	Watermarks with bounded out-of-orderness of 20s for calendar sources; late events beyond 10 minutes trigger a reconciliation recompute rather than silent drop
State	RocksDB state backend, incremental checkpoints every 30s to object storage; exactly-once sinks to the online store via idempotent upserts keyed on <code>(entity_key, feature_view, event_ts)</code>
Reconciliation	Nightly full recompute from provider APIs; drift between streamed and recomputed values is a monitored metric with a 0.5% alert threshold
Backfill	The same Flink job graph runs in batch mode over Iceberg history to produce historical feature values; one codebase for both paths

Webhook-based calendar sync is lossy in practice — expired subscriptions, missed notifications, provider throttling. The architecture therefore treats webhooks as a latency optimisation over a periodic authoritative poll, never as the source of truth. The slot engine re-validates against the provider immediately before placing a hold, so a stale feature can degrade ranking quality but can never cause a double booking.

7. Apache Flink — Stateful stream processing — <https://apache.google.com/flink/master/docs/content/docs/concepts/stateful-stream-processing.md>

8.5 Online and Offline Stores

Store	Technology	Layout	Access pattern
Online	Amazon DynamoDB, with ElastiCache for Redis as a read-through tier for the hottest views	Partition key <code>tenant_id#entity_id</code> , sort key <code>feature_view#version</code>	Batched gets of 20-80 clinician rows per ranking call; p99 <= 12ms at the store
Offline	Apache Iceberg tables on S3 with Glue catalog	Partitioned by <code>tenant_bucket</code> , <code>event_date</code> ; sorted by <code>entity_key</code> , <code>event_ts</code>	Point-in-time joins for training; time travel for reproducibility
Registry	Feast registry backed by PostgreSQL, definitions in Git	Feature views, entities, data sources, owners, PHI class, SLOs	CI validates definitions; changes are pull requests with owner approval

8.6 Point-in-Time Correctness

Every training row is materialised with an as-of join: for a decision at time t , each feature takes the latest value with `event_ts <= t - serving_lag`, where `serving_lag` is the measured p95 staleness of that view in production. Using the ideal as-of time rather than the realistic one is the most common source of train-serve skew; the model learns from fresher features than it will ever see. Labels are joined with their own event times and a label maturity window — a showed label is final only after the scheduled session time plus 48 hours, and a `retained_90d` label only after 90 days — so immature rows are excluded rather than mislabelled as negatives.

8.7 Request-Time Feature Computation

Features that depend on the current inquiry are computed as on-demand feature views with the same transformation code in training and serving.⁵ Examples: overlap between the prospect's stated availability windows and each clinician's open slots, weighted by recency of the slot data; semantic affinity between the extracted concern embedding and the clinician profile embedding; payer-panel compatibility as a tri-state (in-network, out-of-network accepted, excluded). The transformation functions are pure, versioned, and unit-tested against fixtures shared by the training pipeline.

8.8 Governance and PHI Classification

Every feature carries a data class in the registry: `PUBLIC`, `OPERATIONAL`, `SENSITIVE_OPERATIONAL`, `PHI_DERIVED` or `PROHIBITED`. Identity attributes of clinicians are self-declared, consented and used only as a match criterion when the prospect has expressed a preference; they are never used as a general ranking signal. Prospect demographic attributes are `PROHIBITED` as ranker inputs. The registry enforces this at materialisation time — a model training job whose feature list contains a prohibited feature fails before a single row is read. This is how the platform operationalises nondiscrimination obligations for patient-care decision support rather than relying on reviewer diligence.⁸

5. Feast — Feature view concepts — <https://docs.feast.dev/getting-started/concepts/feature-view>

8. Federal Register — Nondiscrimination in Health Programs and Activities (Section 1557) — <https://www.federalregister.gov/documents/2024/05/06/2024-08711/nondiscrimination-in-health-programs-and-activities>

8.9 Monitoring

Signal	Method	Threshold
Freshness	Max now - event_ts per view and tenant	Per-view SLO; page on 3x breach
Null and default rate	Per feature, per tenant	Alert on +5 points versus 7-day baseline
Distribution drift	Population stability index and Jensen-Shannon distance, training vs serving	PSI > 0.2 raises a model review
Train-serve skew	Log served feature vectors; recompute offline for a 1% sample; compare	Mean absolute difference over tolerance per feature
Stream vs batch reconciliation	Nightly diff	> 0.5% mismatch

9 Model Portfolio

The platform runs a portfolio, not a model. Foundation models are commodities behind a gateway; the proprietary performance of the platform comes from eight model families trained on the platform's own decision and outcome data.

ID	Model	Type	Why custom rather than prompted	Serving target
M1	Inquiry understanding encoder	Multi-task transformer encoder (multi-label concern taxonomy, payer span, modality, availability, trait preference, urgency)	20-40x cheaper and 50x faster than an LLM per inquiry; calibrated per-label probabilities; deterministic	Triton + TensorRT FP16, dynamic batching; p95 <= 60ms
M2	Safety sentinel	Multilingual classifier ensemble with lexicon and rule layer	Recall floor must be guaranteed and measured per language; LLM refusal behaviour is not a safety system	Triton + TensorRT, dedicated instance group; p95 <= 150ms
M3	Practice voice adapters	Per-tenant LoRA adapters on an open-weight instruct model, SFT then DPO	"Trained on the practice's voice" at a per-message cost a frontier model cannot match; tenant-scoped memorisation boundary	Multi-LoRA serving on vLLM
M4	Clinician ranker	Gradient-boosted learning-to-rank (LambdaRank objective) over feature-platform vectors	Outcome-optimised, explainable, stable, cheap; LLMs are poor rankers over structured capacity features	Triton FIL backend; p95 <= 25ms for 80 candidates
M5	Engagement and no-show models	Gradient-boosted classifiers and discrete-time survival model for intake completion	Drives nurture cadence and reminder timing	Batch and online
M6	Demand forecaster	Global gradient-boosted model over hierarchical series (tenant x location x specialty), with reconciliation	Hiring and waitlist signals; the "+2 recommended hires" output	Batch hourly
M7	Concern-profile bi-encoder	Contrastively fine-tuned embedding model mapping inquiry concerns and clinician profiles into one space	Off-the-shelf embeddings do not separate EMDR-trauma from EFT-couples specialisation reliably	Batch for profiles, online for inquiries
M8	Reranker and verifier distillates	Small cross-encoder reranker for retrieval; distilled judge for tone and policy conformance	Cuts verifier cost on the hot path by an order of magnitude	Triton (ONNX/TensorRT); p95 <= 80ms

Anything not in this table is served by a foundation model through the gateway: complex multi-turn drafting, document understanding of referral packets, RCM correspondence, directory copy, digest narration and the LLM judge on the safety ambiguous band.

9.1 Matching Ranker in Depth

Matching is the decision with the highest business leverage and the highest fairness exposure, so it gets the most rigorous treatment.

Stage 0 — hard eligibility (deterministic). License jurisdiction and telehealth eligibility for the prospect's location; panel participation or tenant out-of-network policy; age band served; modality offered; language when the prospect requires it; caseload ceiling; clinician-declared exclusions. Eligibility is code, not a model, and its output is auditable: every excluded clinician has a reason code.

Stage 1 — candidate scoring (learned). A LambdaRank model scores eligible clinicians on features from `clinician_capacity_rt`, `clinician_profile`, `clinician_performance_90d`, `clinician_payer_panel` and the request-time affinity features.⁹ Relevance labels are graded outcomes: 0 not selected, 1 consult booked, 2 first session attended, 3 retained at 90 days. Optimising for the graded label rather than for "booked" is the difference between filling slots and making right-fit matches — the product positioning explicitly distinguishes right-fit from first-available, and the label definition is where that distinction is enforced.

Stage 2 — constraint-aware re-ranking. A deterministic re-ranker applies capacity balancing (soft penalty as `caseload_pct` approaches ceiling), exposure fairness for onboarding clinicians within a bounded budget, and stated prospect preferences as hard promotions. The re-ranker is where business policy lives, so that policy changes do not require retraining.

Stage 3 — calibration and explanation. Displayed match percentages are calibrated probabilities of a good outcome, fit with isotonic regression per tenant cohort, not raw ranker margins. A "96%" that is not calibrated is a liability in a regulated context. Explanations are generated from SHAP contributions mapped to human-readable factors (specialty, insurance, availability overlap, modality) and rendered by a small LLM constrained to those factors.

Learning from overrides. The product allows staff to override match scores at any time. Every override is logged with the full ranked list shown, positions and the chosen clinician. This is biased feedback — staff see the top of the list first — so training uses inverse propensity weighting with position propensities estimated from a small, controlled randomisation of positions 2-4 on a fraction of low-risk traffic. Without propensity correction, the ranker learns to reproduce its own past ordering.

9.2 Practice Voice in Depth

The Assistant is trained on each practice's voice. The implementation is a tiered strategy, selected per tenant by data volume and quality.

Tier	Condition	Method
V0 — Style profile	New tenant, fewer than ~300 approved messages	Retrieval of approved exemplars plus a structured style card (formality, warmth markers, sign-off conventions, phrases to avoid) injected by the context compiler
V1 — SFT adapter	~300-1,500 approved, de-identified messages	LoRA SFT on the tenant corpus over a shared, platform-tuned base ¹⁰
V2 — Preference-tuned adapter	> 1,500 messages and > 400 draft-edit pairs	SFT followed by DPO on pairs (staff-edited final preferred over model draft) ¹¹

The key signal is the edit. When a coordinator changes a draft before sending, the pair (model draft, human final) is a direct preference judgement in the tenant's own voice. DPO over those pairs is cheaper and more stable than training a reward model, and it optimises exactly the metric the business cares about — how much a human has to change before sending. Pairs are filtered: edits that change facts (a corrected time, a different clinician) are routed to the factual-error dataset, not the preference dataset, because they are corrections, not stylistic preferences.

The base model for adapters is an open-weight instruct model in the 7-9B class, first adapted by the platform team on a cross-tenant, de-identified, contractually permitted corpus of operational messaging (never clinical

9. LightGBM parameters (ranking objectives) — <https://lightgbm.readthedocs.io/en/latest/Parameters.html>

10. Hu et al., LoRA: Low-Rank Adaptation of Large Language Models — <https://arxiv.org/abs/2106.09685>

11. Rafailov et al., Direct Preference Optimization — <https://arxiv.org/abs/2305.18290>

content) to learn the domain register — acknowledgement, insurance explanation, scheduling logistics, gentle nudges. Tenant adapters are rank 16-32 on attention and MLP projections, typically 20-80MB each, trained with QLoRA on a single accelerator in under an hour.¹² They are served with multi-adapter batching on vLLM, where PagedAttention and adapter hot-swap let several hundred tenant adapters share one base model deployment.¹³

9.3 Safety Sentinel in Depth

The sentinel is a layered ensemble because no single detector achieves the required recall across languages and phrasings.

Layer	Mechanism	Role
L1	Curated multilingual lexicon and pattern rules, clinically reviewed	Guaranteed catch of explicit statements; zero-latency
L2	Fine-tuned multilingual encoder, multi-label (self-harm, harm to others, abuse, medical emergency, minor at risk)	Paraphrase and indirect expression coverage
L3	LLM judge with a clinically authored rubric, invoked only when L2 probability falls in the ambiguous band	Context-dependent disambiguation
L4	Conversation-level escalation tracker	Detects rising risk across messages, not only per message

The decision threshold is set to achieve recall ≥ 0.99 per supported language on a clinically adjudicated golden set, accepting the resulting precision; false positives cost staff attention, false negatives cost lives. Any positive or indeterminate verdict suppresses generative outbound, sends a pre-approved message containing crisis resources — including 988 for U.S. contacts — and pages the tenant's designated on-call with an SLA.¹⁴ The sentinel never attempts to counsel. Low-resource languages below the recall floor are explicitly capped at human-review-only autonomy.

12. Dettmers et al., QLoRA: Efficient Finetuning of Quantized LLMs — <https://arxiv.org/abs/2305.14314>

13. Kwon et al., Efficient Memory Management for LLM Serving with PagedAttention (vLLM) — <https://arxiv.org/abs/2309.06180>

14. SAMHSA — 988 Suicide and Crisis Lifeline — <https://www.samhsa.gov/mental-health/988>

10 Custom Training Pipeline

10.1 Principles

The training pipeline is designed around five commitments that follow from operating on healthcare-adjacent data across many tenants.

- 1. **Training data is a governed product.** Every dataset has an owner, a schema, a data class, a legal basis, a retention policy and a lineage record back to the source events and tenants.
- 2. **De-identification precedes everything downstream of the lake.** No training job reads raw conversation text; it reads de-identified text produced by a versioned de-identification pipeline with measured recall.
- 3. **Tenant boundaries survive training.** Cross-tenant models learn only from de-identified, contractually permitted data; tenant-specific learning lives in tenant-scoped artifacts that can be deleted.
- 4. **Evaluation is a gate, not a report.** A model with a worse safety recall or a subgroup regression does not ship regardless of its aggregate gain.
- 5. **Every run is reproducible.** Code commit, container digest, dataset snapshot ID, feature view versions, hyperparameters and random seeds are recorded; any production model can be rebuilt bit-for-bit or within stated tolerance.

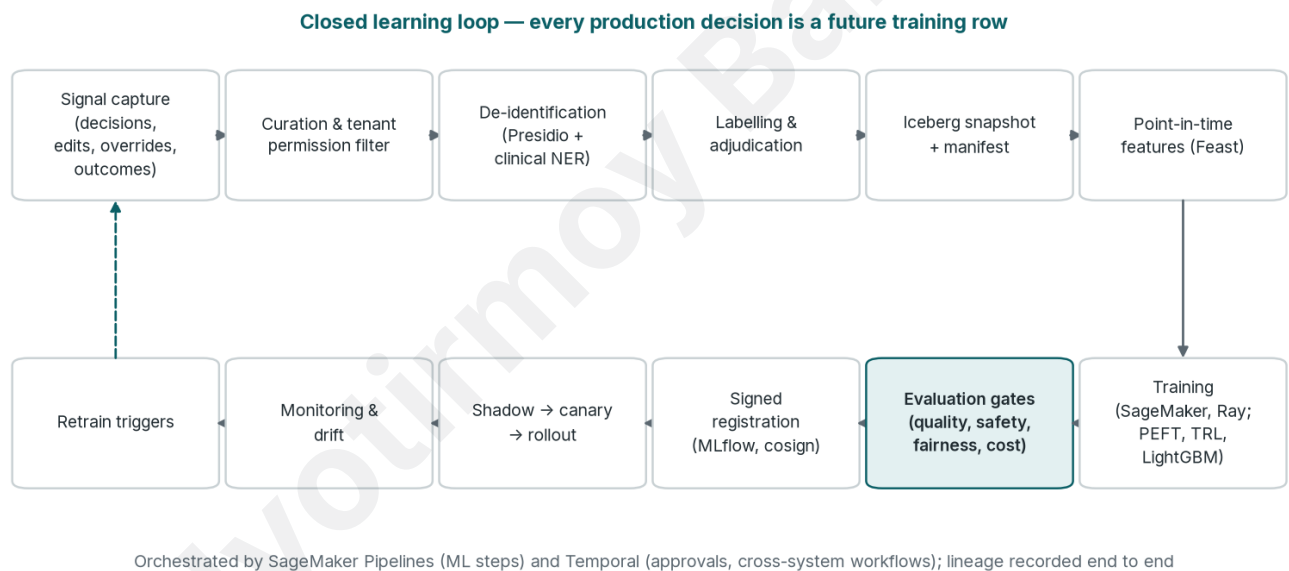


Figure 4. Custom training pipeline — the closed learning loop

10.2 Pipeline Stages

Stage	Function	Technology	Gate
1. Signal capture	Decisions, proposals, human edits, overrides, approvals, rejections and outcomes emitted as events with trace and model version references	Event log, OpenTelemetry trace IDs ¹⁵	Schema registry compatibility
2. Curation	Tenant permission filter, dedup, quality filters, stratified sampling by tenant, language, intent and outcome	Spark on EMR or Glue over Iceberg	Row-level provenance present
3. De-identification	Named-entity and pattern detection of PHI with custom recognisers for payer member IDs, clinician names, practice names, addresses; consistent surrogate replacement	Microsoft Presidio with custom recognisers plus a fine-tuned clinical NER model ¹⁶	Measured PHI recall ≥ 0.995 on the audit set; sampled human audit
4. Labelling	Clinical adjudication for safety sets; ops SME labelling for extraction; weak supervision from outcomes	Label Studio (self-hosted in VPC) with consensus and adjudication workflows	Inter-annotator agreement (Krippendorff alpha ≥ 0.8 for safety)
5. Dataset versioning	Immutable snapshots with manifests	Iceberg snapshots and tags; manifest in MLflow	Snapshot ID pinned in run config
6. Feature materialisation	Point-in-time training sets	Feast historical retrieval over Iceberg ⁵	Skew test against served-feature log
7. Training	Encoders, rankers, forecasters, adapters, distillates	SageMaker Training and Ray Train on EKS; Hugging Face Transformers, PEFT and TRL; LightGBM ¹⁷	Resource and cost budget per job
8. Evaluation	Offline metrics, subgroup analysis, safety suites, red-team suites, regression against champion	MLflow evaluation, custom harness, LLM judges calibrated against human labels ¹⁸	Release gates below
9. Registration	Signed artifact with model card, data card, eval scorecard and lineage	MLflow Model Registry; artifact signing with Sigstore cosign	Approval by model owner and, for safety-relevant models, the clinical safety lead
10. Deployment	Shadow, canary, progressive rollout	NVIDIA Triton on EKS for encoders, rankers and distillates; vLLM on EKS for generative adapters; Bedrock Custom Model Import for managed serving of imported open-weight models ¹⁹	Online guardrail metrics
11. Monitoring and retrain triggers	Drift, quality, outcome decay, label maturity	Evidently-style drift jobs, MLflow traces, CloudWatch and Prometheus	Scheduled and trigger-based retraining

Orchestration of the pipeline is SageMaker Pipelines for managed ML steps and Temporal for cross-system, long-running dataset and approval workflows.^{20,1} Using the same durable workflow runtime that runs the product for the model factory's approval chains is deliberate: model promotion is a business process with humans in it, and it deserves the same durability and audit.

1. Temporal — Workflow Execution — <https://docs.temporal.io/workflow-execution>

5. Feast — Feature view concepts — <https://docs.feast.dev/getting-started/concepts/feature-view>

15. OpenTelemetry GenAI semantic conventions — <https://opentelemetry.io/docs/specs/semconv/gen-ai/>

16. Microsoft Presidio — <https://github.com/microsoft/presidio>

17. Ray Train overview — <https://docs.ray.io/en/latest/train/overview.html>

18. MLflow — LLM and agent evaluation — <https://mlflow.org/docs/latest/genai/eval-monitor/>

19. Amazon Bedrock Custom Model Import — <https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-import-model.html>

20. Amazon SageMaker Pipelines — <https://aws.amazon.com/sagemaker/ai/pipelines/>

10.3 Training Recipes

Model	Recipe	Key details
M1 encoder	Multilingual encoder (mDeBERTa-v3 or XLM-R class) fine-tuned multi-task with shared trunk and per-task heads; focal loss for rare concern labels; temperature scaling per head	Teacher labels from a frontier LLM on 200k de-identified inquiries, then human-corrected 15k gold set; distillation cuts serving cost while keeping the LLM as fallback for low confidence
M2 sentinel	Same encoder family, separate weights; class-balanced sampling; hard-negative mining from L3 disagreements; per-language threshold tuning	Clinically adjudicated gold set is never used for training, only for gating
M3 adapters	Base domain SFT (full or high-rank LoRA) on cross-tenant de-identified corpus; per-tenant QLoRA SFT; DPO with beta tuned per tenant on held-out edit-distance	Adapter retraining weekly for V2 tenants; canary on 10% of that tenant's drafts
M4 ranker	LightGBM LambdaRank, NDCG@3 objective; IPS-weighted; monotone constraints on in-network and availability-overlap features	Monotone constraints prevent pathological inversions and make the model defensible
M5 engagement	LightGBM binary and discrete-time hazard model; calibration by tenant cohort	Drives send-time optimisation for reminders
M6 forecaster	Global LightGBM on lagged, calendar and holiday features, with MinT reconciliation across the hierarchy; quantile outputs P10/P50/P90	Hiring signal uses P50 demand against P80 capacity to avoid overreaction
M7 bi-encoder	Contrastive fine-tune with in-batch negatives plus mined hard negatives (same specialty family, different modality)	Evaluated on recall@10 of eventual booked clinician
M8 distillates	Knowledge distillation from frontier judge to a small cross-encoder on judge-labelled pairs with human spot-checks	Distillate agreement with teacher ≥ 0.9 Cohen's kappa before hot-path use

10.4 Tenant Data Governance in Training

Scenario	Rule
Cross-tenant base models	Only de-identified operational text and structured features from tenants whose agreements permit service-improvement use; opt-out honoured at snapshot time
Tenant adapters	Trained only on that tenant's approved messages; stored encrypted with the tenant's data key; never merged into shared weights
Tenant offboarding	Adapters crypto-shredded; tenant rows removed from future snapshots; base model retrained on the next scheduled cycle without that tenant's contribution; lineage record proves exclusion
Individual deletion request	Record-level lineage identifies affected snapshots; rows tombstoned; affected tenant adapters retrained; cross-tenant models rely on de-identification plus next-cycle retrain
Clinical content	Session notes and clinical records from EHRs are never used for training; the EHR remains the clinical system of record and the platform synchronises operational data only

10.5 Release Gates

Gate	M1	M2	M3	M4
Primary metric	Macro-F1 $\geq$ champion - 0.5pt	Recall $\geq$ 0.99 per language; no regression	Edit distance $\leq$ champion; policy conformance $\geq$ 99.5%	NDCG@3 $\geq$ champion; offline IPS estimate of good-outcome rate $\geq$ champion
Safety	Urgency-label recall $\geq$ 0.97	Zero misses on the critical subset	Zero crisis-content generation on red-team set	No eligibility violations (must be zero by construction)
Fairness	Per-language F1 gap $\leq$ 3pt	Per-language recall floor	Tone parity across languages	Exposure and outcome parity for onboarding clinicians within tolerance
Latency and cost	p95 $\leq$ 60ms	p95 $\leq$ 150ms	Tokens per draft $\leq$ champion + 5%	p95 $\leq$ 25ms
Online	Shadow agreement $\geq$ 97%	Shadow, then canary with human audit of all positives	10% canary per tenant, 7 days	Interleaving test, then 20% A/B with guardrails

10.6 Retraining Triggers

Trigger	Condition	Action
Scheduled	Weekly for adapters and engagement models; monthly for ranker and encoder; quarterly for bi-encoder	Full pipeline run, champion-challenger
Drift	PSI > 0.2 on top-10 features, or embedding centroid shift beyond threshold	Retrain candidate and review
Outcome decay	Rolling good-outcome rate for ranked matches below control limit for 14 days	Ranker investigation and retrain
Human signal	Rejection rate or edit distance for an intent rises beyond control limit	Autonomy demotion (immediate) and adapter retrain
Safety	Any confirmed false negative in production	Incident; hard-negative added; emergency retrain and re-gate

11 AI Control Plane

11.1 Model Gateway

Every model call — LLM, embedding, ASR, reranker, classifier — traverses the gateway. Services hold no provider credentials. The gateway is the enforcement point for five concerns that cannot be delegated to application code: identity and tenant attribution, policy (which data classes may reach which model endpoints under which agreements), budgets and quotas, resilience, and telemetry.

Capability	Implementation detail
Provider abstraction	Unified request schema normalised to the providers in the catalog: Amazon Bedrock (Anthropic Claude, Amazon Nova, Meta Llama), Azure AI Foundry (Azure OpenAI and catalog models) as the provider-independent secondary, self-hosted vLLM for generative adapters, NVIDIA Triton Inference Server for encoders and rankers, and Bedrock Custom Model Import endpoints ^{19,21}
Eligibility	Each endpoint is tagged with its BAA status, data-retention mode, region and permitted data classes; a PHI-bearing request can only route to BAA-covered, zero-retention, in-region endpoints
Resilience	Per-endpoint circuit breakers on error rate and p95 latency; hedged requests for the first-reply path after a p90 deadline; bounded retries with jitter; provider failover within the same capability class
Budgets	Pre-flight token estimation; per-tenant, per-intent and per-workflow ceilings; downgrade-before-reject policy
Caching	Exact-match cache on normalised prompts; provider prefix caching on stable system and policy prefixes; semantic cache on Redis vector similarity restricted to non-PHI knowledge and staff-copilot queries, with tenant, policy-version and data-class in the cache key and a cosine threshold tuned per intent against a false-hit budget of 0.1%
Structured output	JSON-schema constrained decoding where the provider supports it; grammar-constrained decoding on vLLM; validator with typed repair otherwise
Telemetry	OpenTelemetry GenAI semantic conventions for model, tokens, latency, finish reason and cost; prompt and completion bodies stored encrypted, sampled, with PHI-class tags ¹⁵

11.2 Model Router

Routing is a policy, not a hard-coded mapping. The router selects the cheapest endpoint that satisfies the capability, quality floor, latency budget and data-class constraints for a given intent.

Intent class	Default route	Escalation route	Rationale
Extraction and tagging	M1 encoder	Small LLM with schema	Encoder is calibrated and near-free; LLM only when confidence is low
Acknowledgement and routine follow-up	Tenant adapter (M3) on vLLM	Mid-tier hosted LLM	Voice fidelity at the lowest cost per message
Complex or emotionally loaded thread	Frontier LLM with BAA	Human	Where nuance matters, capability beats cost
Translation	Mid-tier LLM with glossary and locale policy	Frontier LLM for low-resource languages, then human	Terminology locks and back-translation QA
Verification	Different-family mid-tier model or M8 distillate	Frontier judge	Model diversity reduces correlated failure

15. OpenTelemetry GenAI semantic conventions — <https://opentelemetry.io/docs/specs/semconv/gen-ai/>

19. Amazon Bedrock Custom Model Import — <https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-import-model.html>

21. Amazon Bedrock security and compliance — <https://aws.amazon.com/bedrock/security-compliance/>

Intent class	Default route	Escalation route	Rationale
Document understanding (referrals)	Multimodal frontier LLM	Human	Layout-heavy, low volume, high value
Narration (digest, dashboards)	Mid-tier LLM with numeric grounding	—	Numbers are tool-sourced; model only narrates

Router decisions are logged with the policy version that made them. Router policy changes are evaluated offline by replaying a week of traffic through the candidate policy with recorded outputs, measuring predicted cost and quality deltas before the change is merged.

11.3 Context Compiler

The context compiler is the most important and least glamorous component in the platform. It turns "what does this node need to know" into a minimal, ordered, budgeted, policy-compliant prompt.

1. **Declare.** Each agent manifest declares its context requirements as typed slots: `thread_summary`, `facts`, `practice_policies`, `availability_summary`, `kb_chunks(k≤6)`, `style_card`, `exemplars(k≤3)`.
2. **Resolve.** Slots are resolved from authoritative sources: the operational store, online features, the knowledge index, the style registry.
3. **Minimise.** The PHI minimiser applies the node's data-class allowance. A drafter needs the prospect's first name and requested times; it does not need date of birth, member ID or presenting-concern free text. Fields beyond the allowance are dropped or replaced with typed placeholders that the dispatch layer rehydrates after generation.
4. **Order and pack.** Stable content first — system instructions, policy, style card — to maximise provider prefix-cache hits; volatile content last. Token budget enforced with per-slot priorities and truncation strategies.
5. **Isolate.** Untrusted content (client messages, referral documents, retrieved web or directory text) is wrapped in delimited, labelled blocks with an explicit instruction hierarchy; it is never concatenated into instruction text.
6. **Record.** The compiled context manifest — slot sources, versions, data classes, token counts — is attached to the trace, so any output can be explained by exactly what the model saw.

Placeholder rehydration is how the Assistant avoids seeing PHI it does not need: the model writes "Hi {{prospect.first_name}}, we have an opening with {{clinician.display_name}} on {{slot.local_datetime}}", and the dispatch layer, which is deterministic and audited, fills the values. The model is excluded from identifiers by construction rather than by instruction.

12 Knowledge and Retrieval

12.1 Knowledge Domains

Domain	Content	Owner	Update model
Practice policy	Fees, cancellation policy, telehealth policy, intake requirements, hours, locations	Tenant admin	Versioned, approval-gated publication
Clinician profiles	Specialties, modalities, bios, languages, populations served	Clinician with admin approval	Event-driven reindex
Payer knowledge	Plan normalisation, panel rules, common client-facing explanations	Platform RCM team	Curated, global, versioned
Operational playbooks	Response templates, escalation procedures, crisis resources by region	Platform clinical safety and ops	Clinically reviewed
Directory content	Approved directory copy per clinician and platform	Clinician	Draft-approve workflow

12.2 Retrieval Pipeline

Retrieval is hybrid by default: BM25 over OpenSearch for exact payer names, plan codes and modality acronyms (EMDR, EFT, DBT), dense retrieval over embeddings for paraphrase, reciprocal-rank fusion, then cross-encoder reranking with the M8 reranker. Every retrieval is tenant-filtered before scoring, not after — post-filtering leaks through ranking side channels and wastes recall.

Chunking is structure-aware: policy documents are chunked by heading with the heading path prepended; FAQs are chunked by question-answer pair; clinician profiles are indexed as single records with field-level embeddings. Each chunk carries `tenant_id`, `domain`, `version`, `effective_from`, `approval_state` and `phi_class`. Only `approval_state=PUBLISHED` chunks are retrievable by client-facing agents; draft content is visible only to staff copilots.

12.3 Vector Store Topology

Tenant knowledge lives in pgvector on Aurora PostgreSQL for the long tail of tenants, co-located with the relational data it references, using HNSW indexes on a tenant-partitioned table so every ANN query is pre-filtered by partition.²² Tenants whose corpora or query volumes exceed a threshold (roughly 250k chunks or 20 QPS sustained) move to dedicated OpenSearch k-NN indexes in the same domain that already serves BM25, which keeps hybrid retrieval inside one engine for the heaviest tenants. The decision is economic and operational: most tenants have a few hundred to a few thousand chunks, for which a separate vector database is pure overhead and a second PHI-adjacent processor to govern. The retrieval interface is identical, so migration is a routing change.

22. pgvector — <https://github.com/pgvector/pgvector>

12.4 Embedding Pipeline

Embeddings are produced by an event-driven pipeline: `kb.chunk.published`, `clinician.profile.updated` and `inquiry.facts.extracted` events trigger embedding jobs; the embedding model version is part of the vector's metadata; a model upgrade triggers a shadow re-embed into a parallel index, an offline recall@10 comparison, and an atomic alias swap. Vectors are never mixed across embedding versions in one index — mixed-version indexes are a silent recall killer.

12.5 Practice Knowledge Graph and GraphRAG

A Neo4j property graph models the entities that matching, eligibility explanation and referral reasoning depend on: Clinician, Specialty, Modality, Population, Language, Location, Payer, Plan, Referrer, Policy and KBChunk, with typed relationships such as `CREDENTIALLED_WITH {effective_from, status}`, `PRACTICES_AT`, `SPECIALISES_IN {evidence}`, `REFERS_TO {inquiries_90d, show_rate}` and `GOVERNED_BY`. Every node and relationship carries `tenant_id`; global nodes (payers, plans, modality taxonomy) are shared and read-only. The graph is materialised from the operational store by CDC through Kafka Connect and is never the system of record — a graph that is allowed to become authoritative will drift from the scheduling and credentialing truth within weeks.

Neo4j runs as AuraDB Enterprise inside a private link for the long tail and as a self-managed causal cluster on EKS for dedicated cells. Tenant isolation is enforced twice: every Cypher template generated by the query layer is parameterised with `$tenant_id` and validated by a Cypher allow-list parser (no model ever writes free Cypher), and role-based property access restricts sensitive properties per service identity.

GraphRAG is used where vector similarity is structurally the wrong tool — questions whose answer is a path, not a passage.

Question class	Retrieval strategy	Example
Single-fact policy lookup	Hybrid BM25 + dense over KB chunks	"What is the late-cancellation fee?"
Eligibility explanation	Graph path query, then chunk hydration	"Why is Dr. Reyes eligible for this BCBS PPO client?" returns the credentialing path and effective date
Multi-hop operational reasoning	Iterative multi-hop retrieval: entity linking to seed nodes, bounded k-hop expansion ($k \leq 3$), community summaries, then dense re-retrieval of supporting chunks	"Which referrers send adolescent anxiety cases that we lose for lack of evening in-network capacity?"
Network-level insight	Pre-computed community summaries over the referral graph (Leiden clustering), refreshed nightly	Weekly digest narratives on referral momentum

Multi-hop retrieval is bounded by hop count, node budget and token budget, and every hop is recorded in the context manifest so an answer can be traced to the exact subgraph it used. Community summaries are generated offline by a mid-tier model and stored as versioned `Summary` nodes with provenance edges to their member nodes, which gives data lineage from a narrative sentence back to the operational facts that produced it.

13 Speech and Multilingual Intelligence

13.1 Voice Pipeline

The voicemail-in-60-seconds commitment sets the pipeline shape.

Step	Budget	Detail
Media capture	Real time	Telephony provider records to encrypted object storage via a HIPAA-eligible configuration; recording event emitted
Chunked ASR	~0.3x real time	Streaming ASR with custom vocabulary (clinician names, payer names, modalities); language ID on first 5 seconds
Diarisation and redaction	+3s	Speaker separation for call recordings; PHI span tagging for downstream minimisation
Early routing	At ~15s of transcript	Safety sentinel and understanding run on partial transcript; routing decision can precede full transcript
Summarisation	+4s	Structured summary: reason for call, callback window, payer, concerns, urgency
Reconciliation	+2s	Final transcript supersedes partial; tags re-computed; routing corrected if changed

Early routing on partial transcripts is what makes 60 seconds achievable for long voicemails. The reconciliation step exists because partial-transcript decisions are occasionally wrong; the system corrects itself visibly rather than silently.

13.2 Multilingual Pipeline

The product commits to 30+ languages auto-detected and threads that continue in the detected language with consistent voice and branding. Detection is the easy part. The architecture treats languages in tiers.

Tier	Languages	Autonomy	Quality controls
L-A	English, Spanish	Up to A3 for approved intents	Native-speaker golden sets; per-language adapters where volume allows
L-B	High-resource languages with adequate eval coverage	A3 for templated intents, A1 for free text	Back-translation consistency, glossary conformance, sampled native review
L-C	Detected but low-resource	A1 only; human approves every message	Translation with explicit uncertainty; human routing to bilingual staff

Localised templates for acknowledgements and reminders are pre-translated and human-approved; generated content is used only where templates do not fit. Terminology locks ensure clinician names, practice names, payer names and appointment details are never translated or transliterated inconsistently.

14 LLMOps, Observability and AI SRE

14.1 Artifact Inventory

Every artifact that can change behaviour is versioned, owned and deployable independently: system prompts, policy bundles, agent manifests, graph definitions, router policies, guardrail configurations, retrieval configurations, feature view definitions, model weights and adapters, and evaluation datasets. A production response is fully explained by a tuple of these versions, recorded on its trace.

14.2 Tracing Model

Each inquiry lifecycle is one distributed trace family: a root span per workflow run, child spans per activity, per graph node, per model call and per tool call, following the OpenTelemetry GenAI semantic conventions.¹⁵ Traces are exported through OpenTelemetry collectors to Amazon CloudWatch and Grafana on the primary cloud and to Azure Monitor / Application Insights for the Azure failover inference path, so a request that fails over across providers remains one trace with PHI-bearing attributes encrypted and access-controlled. Traces double as evaluation assets: MLflow ingests sampled traces to build regression datasets and to run offline judges.¹⁸

14.3 AI SLOs

SLO	Objective	Measurement window
First-reply latency	p50 <= 90s, p95 <= 150s, business-hours and after-hours measured separately	28 days
Voicemail routing	p95 <= 60s from recording end	28 days
Safety sentinel availability	99.99%; fail-closed on unavailability	28 days
Draft acceptance	>= 85% approved without edits for A3 intents, per tenant	7 days rolling
Grounded-claim rate	>= 99.5% of factual claims in outbound drafts traceable to state or approved KB	Sampled daily
Feature freshness	Per-view SLOs as defined in the feature platform	7 days
Cost per inquiry	Within +/- 10% of plan, per tenant cohort	Monthly

Error budgets are enforced: when the draft acceptance SLO burns faster than its budget for a tenant, autonomy is demoted automatically, and release of new voice adapters for that tenant is frozen until the budget recovers.

14.4 Incident Classes

Class	Example	Response
AI-SEV1	Safety false negative; cross-tenant data exposure; unauthorised outbound	Kill switch per intent or fleet-wide; human-only mode; incident commander; regulatory assessment
AI-SEV2	Systematic wrong language, wrong clinician names, factual errors on appointment times	Roll back offending artifact version; replay affected threads; client correction messages
AI-SEV3	Tone drift, elevated edit distance, cost anomaly	Demote autonomy; fix forward

15. OpenTelemetry GenAI semantic conventions — <https://opentelemetry.io/docs/specs/semconv/gen-ai/>

18. MLflow — LLM and agent evaluation — <https://mlflow.org/docs/latest/genai/eval-monitor/>

Kill switches exist at four granularities — fleet, tenant, intent and model endpoint — and are exercised quarterly in game days.

Jyotirmoy Bardhan

15 Resilience, Disaster Recovery and Continuity

15.1 RTO and RPO by Capability

Recovery objectives are set per capability, because a single platform-wide RTO either overspends on analytics or underprotects the crisis path.

Capability tier	Examples	RTO	RPO	Topology
C0 — Safety-critical	Safety sentinel, crisis escalation, on-call paging	<= 5 min	0 (stateless; verdicts re-computable)	Active-active across two AWS regions; template-only crisis path survives total AI loss
C1 — Lifecycle core	Inquiry capture, conversation, scheduling holds, dispatch	<= 60 min (region loss); <= 5 min (AZ loss)	<= 5 min	Warm standby in the secondary AWS region; Aurora Global Database; MSK cross-region replication
C2 — Intelligence	Drafting, matching, extraction	<= 15 min degraded, <= 60 min full	n/a (models are artifacts)	Cross-provider failover (Bedrock to Azure AI Foundry); template and human-queue degraded modes
C3 — Learning and analytics	Training, lakehouse, dashboards, digest	<= 24 h	<= 1 h	Pilot light; Iceberg data replicated by S3 cross-region replication

15.2 Hybrid DR and Failover Topology

The topology is hybrid on two axes: across AWS regions for state, and across clouds for inference.

Layer	Primary	Secondary	Failover mechanism
Edge and DNS	Route 53 with health-checked records, CloudFront	Secondary-region API endpoints	Health-check failover; tenant pinning preserved by cell ID
Compute	EKS in primary region, multi-AZ	EKS in secondary region at 25% warm capacity, Karpenter scale-out	Argo CD applies the same GitOps state; Karpenter provisions to target within ~10 minutes
Workflows	Temporal cluster in primary	Temporal multi-cluster replication (namespace failover)	Namespace failover; in-flight workflows resume from replicated history
Operational data	Aurora PostgreSQL Global Database	Secondary cluster, typical replication lag < 1s	Managed planned or unplanned failover; application reconnects through a cell-aware proxy
Events	Amazon MSK	MSK in secondary via MirrorMaker 2	Consumers resume from translated offsets; idempotent handlers absorb duplicates
Online features	DynamoDB global tables	Replica in secondary	Multi-region active reads; freshness SLO temporarily relaxed
Foundation models	Amazon Bedrock (primary region, cross-region inference profiles)	Azure AI Foundry under BAA, same capability classes	Router policy flips capability classes per circuit-breaker state; no code change
Self-hosted models	vLLM and Triton on EKS GPU pools	Secondary-region GPU pool at minimum warm size; hosted fallback	Adapters and engines stored in replicated S3; cold start of an adapter < 90s
Secrets and keys	AWS KMS multi-region keys, Secrets Manager replication	Secondary replicas	Automatic

Azure is deliberately scoped to inference failover and a subset of observability, not to a full second data plane. A full active-active multi-cloud data plane would double the compliance surface for PHI storage and triple the operational burden for a benefit — surviving the simultaneous loss of two AWS regions — that the risk model did not justify. That was the single most expensive decision I declined, and the reasoning is recorded in the trade-off register.

15.3 Continuity Lifecycle Orchestration

Failover is itself a workflow. A Temporal ContinuityRunbook workflow executes the region-failover sequence — freeze deploys, verify replication lag, promote Aurora secondary, fail over Temporal namespaces, flip DNS, scale secondary compute, re-point feature reads, re-validate calendar and EHR connector health, lift freeze — with human approval signals at the two irreversible steps and automatic rollback on failed health gates. Encoding the runbook as a workflow means every exercise produces a timed, auditable history, and the measured RTO is a number from the last run, not an estimate.

15.4 Circuit Breakers and Graceful Degradation

Dependency	Breaker signal	Degraded behaviour	Customer-visible effect
Frontier LLM endpoint	Error rate > 5% or p95 > 2x baseline over 60s	Route to alternate provider in the same capability class	None
All generative endpoints	Both providers tripped	Pre-approved localised templates for acknowledgements; drafts queued for humans	Replies are templated, not personalised
Safety sentinel model	Unavailable	Lexicon layer only, all messages treated as elevated, human review mandatory	Slower replies; no reduction in safety
Calendar provider	Webhook silence or API 429s	Authoritative poll at reduced frequency; slot offers marked "to be confirmed"	Confirmation may take longer
EHR connector	Sync failures	Queue handoffs with reconciliation tasks	Delayed EHR appearance
Vector or graph store	Timeouts	Lexical-only retrieval; graph explanations omitted	Less detailed explanations

15.5 Resilience KPIs and Simulation-Based Validation

KPI	Target	Instrumentation
Measured region-failover RTO (C1)	<= 60 min	ContinuityRunbook workflow duration from last game day
Measured RPO (C1)	<= 5 min	Aurora Global Database lag and MSK replication lag at failover instant
Provider failover time (C2)	<= 2 min to full alternate routing	Router policy state change to first successful alternate response
Degraded-mode correctness	100% of crisis-path test messages escalated during degraded mode	Synthetic canary messages injected every 5 minutes
Mean time to detect AI regressions	<= 15 min	Burn-rate alerts on draft acceptance and verifier veto rate

Resilience is validated by simulation, not assertion. Chaos experiments run monthly in staging and quarterly in production under change control, using AWS Fault Injection Service and Chaos Mesh on EKS: AZ loss, Aurora failover, MSK broker loss, LLM provider brown-out (injected latency and 5xx at the gateway), calendar-provider throttling, GPU node pool loss and full-region evacuation. Each experiment has a hypothesis, a steady-state metric and an abort condition; results feed the resilience KPI dashboard and the quarterly continuity review.

Jyotirmoy Bardhan

16 Core Tech Stack

16.1 Selection Method

Technology selection was the architectural decision I spent the most time defending, because it is the decision that is hardest to reverse. Every candidate was scored on seven weighted criteria specific to this platform, and every contested choice was settled by a time-boxed spike against a pre-agreed metric rather than by preference.

Criterion	Weight	What it measured
Regulatory fit	25%	BAA availability, HIPAA eligibility, data residency, retention and zero-data-retention controls, auditability
Tenant isolation primitives	15%	Native support for row-, partition-, namespace-, key- and network-level tenant separation
Tail latency on the first-reply path	15%	p95 and p99 behaviour under bursty, low-QPS load; cold-start characteristics
Operational burden	15%	On-call load and specialist skills required for a platform team of 28 engineers
Portability and lock-in exposure	10%	Cost of exit at the layers where lock-in is most expensive: gateway, orchestration, feature definitions, model formats
Unit cost at planning load	10%	Three-year TCO at the planning load model, including people cost
Ecosystem maturity	10%	Production references, release cadence, hiring market

The platform is AWS-primary because its HIPAA-eligible managed surface covers the most of this stack with the least undifferentiated operations, and Azure is the provider-independent secondary for inference and disaster recovery. Portability is preserved deliberately at four layers — model gateway, workflow and agent orchestration, feature definitions and model artifact formats — and accepted as coupling everywhere else.

16.2 Competency-to-Platform Mapping

The following maps each capability area in the reference competency list to its disposition on this platform:

Adopted (in the production architecture), **Adopted — scoped** (used for a defined subset of the platform) or **Evaluated — not adopted** (assessed, with the reason it lost).

Agentic AI and Multi-Agent Systems

Capability	Disposition	Where and why
Multi-agent orchestration	Adopted	Supervised hierarchy of 15 agents inside durable workflows; typed handoffs, asymmetric safety barrier, generate-verify-repair
Agent-to-Agent (A2A) communication	Adopted — scoped	Organisational boundary only: partner billing and care-coordination agents; internal agents use typed function calls ⁴
Model Context Protocol (MCP)	Adopted	Every internal capability exposed as an MCP server behind the tool gateway ³
LangGraph	Adopted	Agent graph runtime with typed state, reducers, interrupts and a PostgreSQL checkpoint ²
Checkpoint/resume state management	Adopted	Two layers: LangGraph checkpoints for reasoning state; Temporal event history for business lifecycle ¹

1. Temporal — Workflow Execution — <https://docs.temporal.io/workflow-execution>

2. LangGraph overview — LangChain docs — <https://docs.langchain.com/oss/python/langgraph/overview>

3. Model Context Protocol — Tools specification — <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>

4. Agent2Agent (A2A) Protocol specification — <https://github.com/a2aproject/A2A/blob/main/docs/specification.md>

Capability	Disposition	Where and why
Provider-independent agent orchestration	Adopted	Agents bind to capability classes, never to model names; tools over MCP, not vendor function formats
Multi-persona control planes	Adopted	Six persona profiles as signed policy bundles parameterising one agent fabric
Tool-calling and schema-validated tool contracts	Adopted	JSON Schema contracts per tool, validated at the gateway; capability tokens scoped to entity and verb

Model Serving and Inference Optimisation

Capability	Disposition	Where and why
NVIDIA Triton Inference Server	Adopted	Serves encoders (M1, M2), rankers (M4, FIL backend) and distillates (M8); one serving plane for non-generative models
Dynamic batching	Adopted	Triton dynamic batcher with preferred batch sizes 8/16/32 and max queue delay 4ms for M1; continuous batching in vLLM for generative models
TensorRT optimisation	Adopted — scoped	TensorRT FP16 engines for encoders and the reranker; TensorRT-LLM evaluated for generative adapters and not adopted, because multi-LoRA hot-swap flexibility outweighed its throughput gain at this load
vLLM	Adopted	Multi-LoRA serving of tenant voice adapters with PagedAttention, prefix caching and chunked prefill ¹³
Model routing (cost, latency, capability)	Adopted	Router policy per intent class with quality floors, latency budgets, data-class eligibility and cost ceilings
Semantic caching	Adopted — scoped	Redis vector similarity cache for non-PHI knowledge and staff-copilot queries only; disabled for PHI-bearing prompts
GPU utilisation tuning	Adopted	MIG partitions for Triton encoders, vLLM memory and concurrency tuning, KEDA scaling on queue depth, warm minimums per pool
Managed vs self-hosted inference analysis	Adopted	Formal three-year TCO and latency analysis per capability class; result: managed for frontier capability, self-hosted for high-volume narrow tasks

Retrieval, Knowledge and Data Architecture

Capability	Disposition	Where and why
GraphRAG	Adopted — scoped	Path-shaped questions: eligibility explanations, referral-network insight, multi-hop operational analysis
Knowledge graph design (Neo4j)	Adopted	Practice knowledge graph, materialised by CDC, never the system of record
Vector search	Adopted	pgvector for the long tail, OpenSearch k-NN for large tenants, hybrid with BM25 ²²
Multi-hop retrieval	Adopted	Bounded k-hop graph expansion with dense re-retrieval; every hop in the context manifest
Embedding pipelines	Adopted	Event-driven, version-stamped, shadow re-embed with alias swap
Event-driven data architecture	Adopted	Tenant-partitioned Kafka log, outbox pattern, CDC, event-sourced aggregates
Data lineage and provenance tracking	Adopted	OpenLineage events from Spark, Flink and training jobs into Marquez; decision-record lineage to model artifacts
Canonical interoperability contracts (FHIR-based)	Adopted	FHIR R4-shaped internal model for EHR connectors and intake ²³

13. Kwon et al., Efficient Memory Management for LLM Serving with PagedAttention (vLLM) — <https://arxiv.org/abs/2309.06180>

22. pgvector — <https://github.com/pgvector/pgvector>

23. HL7 FHIR R4 — <https://hl7.org/fhir/R4/>

Cloud and Distributed Systems Platforms

Capability	Disposition	Where and why
Amazon EKS	Adopted	All services, agents, Triton, vLLM and Ray workloads
Azure AKS	Evaluated — not adopted	The Azure footprint is consumption of managed inference and monitoring; a second Kubernetes estate would add operational load with no workload that requires it
Amazon Bedrock	Adopted	Primary managed foundation models under BAA; Bedrock Guardrails; Custom Model Import ^{21,19}
Azure AI Foundry	Adopted — scoped	Provider-independent secondary for every generative capability class; inference failover target
Amazon MSK (Kafka)	Adopted	Event backbone with Schema Registry and cross-region replication
Azure Event Hubs	Evaluated — not adopted	No Azure-resident producers or consumers; MSK with MirrorMaker 2 covers regional DR
Istio service mesh	Adopted	mTLS, workload identity, AuthorizationPolicy per cell, egress gateway enforcing destination allow-lists
Multi-tenant isolation architecture	Adopted	Isolation at identity, row, partition, key, cache, feature, model, namespace and network layers
5-tier distributed system design	Adopted	Channel and edge, experience and API, orchestration, intelligence, data and knowledge

LLMOps, Observability and Evaluation

Capability	Disposition	Where and why
OpenTelemetry	Adopted	GenAI semantic conventions on every model, tool and agent span ¹⁵
Azure Monitor / Application Insights	Adopted — scoped	Telemetry for the Azure inference failover path, correlated to the same trace IDs
Amazon CloudWatch	Adopted	Infrastructure metrics, managed-service telemetry, alarms driving circuit breakers
Evaluation metrics and offline/online pipelines	Adopted	Node, trajectory and outcome evaluation; trace replay; MLflow GenAI evaluation ¹⁸
Drift detection and mitigation	Adopted	PSI and embedding-centroid drift, outcome decay, automatic autonomy demotion
Resilience KPI instrumentation	Adopted	RTO, RPO, failover time and degraded-mode correctness as dashboards from real runs
Simulation-based failover validation	Adopted	AWS Fault Injection Service and Chaos Mesh experiments, ContinuityRunbook game days

AI Red Teaming and Security Governance

Capability	Disposition	Where and why
PyRIT	Adopted	Automated multi-turn adversarial campaigns against the agent fabric, scored by rubric-based and classifier scorers
Giskard	Adopted	Vulnerability scans for hallucination, injection, harmful content and bias; RAG evaluation test-set generation

15. OpenTelemetry GenAI semantic conventions — <https://opentelemetry.io/docs/specs/semconv/gen-ai/>

18. MLflow — LLM and agent evaluation — <https://mlflow.org/docs/latest/genai/eval-monitor/>

19. Amazon Bedrock Custom Model Import — <https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-import-model.html>

21. Amazon Bedrock security and compliance — <https://aws.amazon.com/bedrock/security-compliance/>

Capability	Disposition	Where and why
Adversarial prompt injection and jailbreak testing	Adopted	Domain attack corpora for direct and indirect injection across all channels and document types
Bedrock Guardrails	Adopted	Denied topics, content filters, PII filters and contextual grounding checks on Bedrock routes; Azure AI Content Safety on Azure routes
Data exfiltration path analysis	Adopted	Formal enumeration of every path from PHI source to external sink, each with a control and a test
Regulatory compliance automation (AI-agent-led audit)	Adopted	Read-only compliance agent collecting evidence against a controls catalogue
Tenant-level network policy enforcement	Adopted	Default-deny NetworkPolicies per cell namespace, Istio AuthorizationPolicy, egress gateway allow-lists

FinOps and Cost Engineering

Capability	Disposition	Where and why
Per-tenant token-cost attribution	Adopted	Every call attributed to tenant, persona, intent, agent, endpoint and workflow run
LLM cost optimisation	Adopted	Small-model-first routing, adapters, prefix caching, context discipline, batch inference
Budget anomaly alerting	Adopted	Spend-velocity anomaly detection per tenant and intent with automatic downgrade
TCO modelling (3-year horizon)	Adopted	Required input for every managed-vs-self-hosted and build-vs-buy decision
Cost-per-session and cost-per-evidence-unit analysis	Adopted	Cost per inquiry lifecycle, per booked first session, and per audit evidence unit
Chargeback modelling	Adopted — scoped	Showback by tenant and tier feeding pricing; no direct customer chargeback under scale-based pricing

Resilience, DR and SRE

Capability	Disposition	Where and why
Hybrid DR/failover topology design	Adopted	Cross-region for state, cross-cloud for inference
Continuity lifecycle orchestration	Adopted	ContinuityRunbook as a Temporal workflow with approval gates
RTO/RPO-driven architecture	Adopted	Objectives per capability tier C0-C3
Chaos-style failover simulation	Adopted	Monthly staging, quarterly production experiments
Circuit breakers and graceful degradation	Adopted	Per-endpoint breakers; templated, human-queue and lexical-only degraded modes

Programming, Platform Engineering, DevOps and Delivery

Capability	Disposition	Where and why
Python	Adopted	AI services, agents, pipelines, training
SQL	Adopted	Operational queries, dbt models, feature definitions, semantic metrics
FastAPI	Adopted	Python service APIs with Pydantic contracts shared with agent tool schemas

Capability	Disposition	Where and why
REST/GraphQL API design	Adopted	REST and gRPC between services; GraphQL BFF for the staff workspace
Event-driven microservices	Adopted	Bounded contexts communicating through the event log and outbox
Redis	Adopted	Exact and semantic caches, rate limits, idempotency keys, hot feature tier
Cosmos DB	Evaluated — not adopted	No Azure-resident state; DynamoDB global tables and Aurora cover the required access patterns
Amazon S3	Adopted	Encrypted object store, Iceberg lakehouse, model and adapter artifacts
Terraform	Adopted	All infrastructure in both clouds
GitHub Actions	Adopted	CI for code, models and prompts; OIDC federation to AWS and Azure
Azure DevOps	Evaluated — not adopted	One CI/CD system across both clouds; GitHub Actions deploys to Azure via OIDC
Architecture Decision Records	Adopted	Every irreversible decision recorded, owned and reviewed
CI/CD for AI systems	Adopted	Prompts, policies, graphs, adapters and models ship through eval-gated pipelines
Kubernetes resource quotas and NetworkPolicies	Adopted	ResourceQuota and LimitRange per cell namespace; GPU quotas per tenant tier; default-deny networking

Other Tech Stacks and Tools

Technology	Why the platform needs it
Temporal	Durable execution for multi-day follow-up cadences, human waits, sagas and the continuity runbook ¹
Feast	Open feature definitions with point-in-time training retrieval and online serving ⁵
Apache Flink (Amazon Managed Service for Apache Flink)	Event-time, exactly-once streaming for capacity features ⁷
Apache Iceberg on S3 with AWS Glue	Time-travel lakehouse for reproducible training sets
Aurora PostgreSQL with pgvector	Relational integrity with row-level security and co-located vectors
DynamoDB global tables	Multi-region online feature store
OpenSearch	BM25, hybrid search and k-NN for large tenants
Debezium and Kafka Connect	CDC into the event log, feature platform and graph
LiteLLM-based model gateway core	Provider-neutral request normalisation extended with platform policy
NVIDIA NeMo Guardrails	Dialogue-level policy rails ²⁴
Microsoft Presidio	PII and PHI detection for de-identification and egress rails ¹⁶
OPA / Cedar	Policy-as-code for ABAC, persona profiles and autonomy
MLflow	Experiment tracking, registry, GenAI evaluation and trace-based datasets
SageMaker Training, SageMaker Pipelines, Ray Train	Managed and distributed training and pipeline orchestration ^{17,20}
PyTorch, Hugging Face Transformers, PEFT, TRL, LightGBM	Adapter, encoder and ranker training ^{10,11,9}
Label Studio	In-VPC labelling and clinical adjudication

1. Temporal — Workflow Execution — <https://docs.temporal.io/workflow-execution>

5. Feast — Feature view concepts — <https://docs.feast.dev/getting-started/concepts/feature-view>

7. Apache Flink — Stateful stream processing — <https://apache.googlesource.com/flink/+master/docs/content/docs/concepts/stateful-stream-processing.md>

9. LightGBM parameters (ranking objectives) — <https://lightgbm.readthedocs.io/en/latest/Parameters.html>

10. Hu et al., LoRA: Low-Rank Adaptation of Large Language Models — <https://arxiv.org/abs/2106.09685>

11. Rafailov et al., Direct Preference Optimization — <https://arxiv.org/abs/2305.18290>

16. Microsoft Presidio — <https://github.com/microsoft/presidio>

17. Ray Train overview — <https://docs.ray.io/en/latest/train/overview.html>

20. Amazon SageMaker Pipelines — <https://aws.amazon.com/sagemaker/ai/pipelines/>

24. NVIDIA NeMo Guardrails developer guide — <https://docs.nvidia.com/nemo/guardrails/latest/home>

Technology	Why the platform needs it
garak and promptfoo	LLM vulnerability probing and prompt regression tests alongside PyRIT and Giskard
Karpenter and KEDA	Node autoscaling including GPU pools; event-driven pod autoscaling
Argo CD and Argo Rollouts	GitOps and progressive delivery for services and model servers
Sigstore cosign	Signing of images, models and adapters
SPIFFE / SPIRE	Workload identity federated into Istio
AWS KMS, Secrets Manager, Macie, GuardDuty, Security Hub	Per-tenant keys, secrets, PHI discovery in the lake, threat detection
AWS Fault Injection Service and Chaos Mesh	Chaos experiments
Grafana (Tempo, Loki, Mimir) and Langfuse	Unified traces, logs, metrics and LLM trace exploration
OpenLineage and Marquez	Data and model lineage
Great Expectations and Evidently	Data contracts and drift
dbt	Governed metrics layer for analytics and the digest narrator
Twilio, Amazon SES, Gmail and Microsoft Graph APIs	SMS, voice and email channels under BAA

Experience & API	TypeScript workspace + GraphQL BFF	FastAPI, gRPC, Go tool gateway	API Gateway, WAF, CloudFront	Twilio, SES, Gmail / Graph			
Orchestration	Temporal	LangGraph + checkpointer	MCP servers	A2A (partners)	Persona profiles		
AI control plane	LiteLLM-core gateway + router	Bedrock Guardrails / Content Safety	NeMo Guardrails + validators	OPA / Cedar	Redis semantic cache		
Models & serving	Amazon Bedrock	Azure AI Foundry	Triton + TensorRT	vLLM multi-LoRA	Bedrock Custom Model Import		
Data, features, knowledge	Aurora PG + pgvector	MSK + Debezium	Managed Flink	Feast + DynamoDB / Redis	S3 + Iceberg	OpenSearch	Neo4j
Model factory	SageMaker, Ray Train	PyTorch, PEFT, TRL, LightGBM	Label Studio, Presidio	MLflow	SageMaker Pipelines		
Trust & red team	PyRIT	Giskard	garak, promptfoo	Compliance agent	Exfil path tests		
Ops, SRE & security	OpenTelemetry	CloudWatch, Grafana, Azure Monitor	FIS, Chaos Mesh	KMS, Macie, GuardDuty	SPIFFE, cosign		
Platform & delivery	EKS, Karpenter, KEDA	Istio + NetworkPolicies	Terraform	GitHub Actions, Argo CD / Rollouts			

Figure 5. Core tech stack by layer

16.3 Stack by Layer: Decisions in Depth

Each layer below records the decision, why it fits this platform, how it is configured, what it beat, the risk it carries and how we would exit it.

Layer 1 — Cloud Foundation and Tenancy Topology

Decision. AWS as primary cloud in a multi-account landing zone (AWS Organizations, Control Tower), with one account per cell, a shared services account for the AI control plane and model factory, a separate log-archive and audit account, and a security tooling account. Azure is attached through a single subscription scoped to Azure AI Foundry, Azure AI Content Safety and Azure Monitor.

Why it fits. Account-per-cell turns tenant isolation for the largest networks into an IAM and billing boundary that auditors understand without explanation, and caps blast radius at the account level. It also makes per-cell cost attribution exact.

Configuration. Service control policies deny non-approved regions, deny disabling CloudTrail, GuardDuty or KMS key deletion, and deny any Bedrock model not on the approved list. Private connectivity to Bedrock through VPC endpoints; to Azure AI Foundry through private endpoints over a site-to-site link.

Beat. A single shared account with tag-based isolation (weaker blast-radius guarantees, harder audit story) and a fully symmetric multi-cloud estate (double compliance surface for PHI at rest).

Risk and exit. Coupling to AWS managed services below the portability layers is accepted. The exit path is documented per layer and tested for the inference tier through live failover.

Layer 2 — Compute, Mesh and Network Isolation

Decision. Amazon EKS per cell, Karpenter for node provisioning (CPU, GPU and Inferentia pools), KEDA for queue-depth autoscaling, Istio for mTLS, authorisation and egress control.

Why it fits. The platform needs GPU scheduling for Triton and vLLM, Ray for training, long-running Temporal workers and fine-grained network policy — a combination only Kubernetes provides coherently.

Configuration. One namespace per bounded context per cell; default-deny Kubernetes NetworkPolicies; Istio PeerAuthentication in STRICT mode; AuthorizationPolicy allowing only declared service-to-service calls; all egress through an Istio egress gateway with an allow-list of external destinations (model endpoints, calendar and EHR APIs, Twilio, email providers). ResourceQuota and LimitRange per namespace; GPU quotas per tenant tier so a single tenant's backfill cannot starve first-reply inference. Pod Security Standards at restricted; images signed and verified at admission.

Beat. ECS Fargate (simpler, but weak for GPU multi-model serving and mesh-level egress control) and Lambda-heavy designs (cold starts incompatible with the latency budget on the model path).

Risk and exit. Istio adds operational complexity; mitigated by a small, dedicated platform SRE ownership and ambient-mode evaluation for sidecar overhead. Kubernetes manifests are portable by definition.

Layer 3 — Experience, API and Service Engineering

Decision. TypeScript staff workspace with a GraphQL BFF; Python services on FastAPI with Pydantic contracts; gRPC for high-volume internal calls; Go for the tool gateway and channel adapters.

Why it fits. Pydantic models are the single source for REST contracts, event schemas and MCP tool schemas, which eliminates drift between what an agent is told a tool accepts and what the service validates. Go gives predictable latency for the policy-heavy tool gateway on the hot path.

Configuration. API Gateway and WAF at the edge with per-tenant rate limits; OpenAPI specs generated from FastAPI and contract-tested in CI; idempotency keys mandatory on every mutating endpoint; outbox pattern for every state change that emits an event.

Beat. A Java/Kotlin service tier (strong, but splits the team's language surface away from the ML stack).

Layer 4 — Orchestration: Workflows, Agents and Tools

Decision. Temporal for durable business lifecycles, LangGraph for agent reasoning graphs executed as Temporal activities, MCP for tools behind a Go tool gateway, A2A for partner agents.

Why it fits. Follow-up cadences run for days, humans take minutes to hours to approve, and failures must compensate. Temporal gives replayable history, timers, signals and sagas; LangGraph gives explicit topology, typed state, interrupts and checkpoints. Keeping them separate means a deploy never loses a conversation and a reasoning bug never corrupts lifecycle state.

Configuration. Temporal namespaces per cell with multi-cluster replication; activity heartbeats and retry policies per activity type; LangGraph PostgreSQL checkpointer co-located with the cell database; graph definitions versioned and replay-tested; MCP servers per domain, each with JSON Schema tool contracts and a declared side-effect class.

Beat. AWS Step Functions (viable for linear flows, awkward for signal-heavy long-lived conversations), managed agent platforms (opaque planning loops, limited trajectory control) and conversational multi-agent frameworks (non-deterministic trajectories and unbounded cost).

Risk and exit. Two runtimes to operate. The exit path is clean because workflow code and graph code are both plain Python against narrow interfaces.

Layer 5 — AI Control Plane: Gateway, Routing, Policy and Guardrails

Decision. A model gateway built on a LiteLLM core and extended in-house with PHI-class routing, persona policy, budgets, hedging and telemetry; OPA/Cedar for policy decisions; a guardrail chain of Bedrock Guardrails (Bedrock routes), Azure AI Content Safety (Azure routes), NeMo Guardrails dialogue rails and custom validators.

Why it fits. The gateway is where the platform's trust promises are enforced. It must know which endpoints are BAA-covered and zero-retention, which persona is calling, what the tenant's budget is and which capability class is requested — none of which a vendor gateway models natively.

Configuration. Capability classes (`draft.practice_voice`, `classify.safety`, `verify.policy`, `translate.localise`, `summarise.call`) mapped to ordered endpoint lists per region; circuit breakers per endpoint; hedged requests on the first-reply path after p90; exact cache and prefix caching always on; semantic cache only for non-PHI classes. Guardrails are ordered cheapest-first and each has a precision, recall and latency budget on a maintained test suite.

Beat. Purely cloud-native gateways (tie routing to one provider) and commercial AI gateways (policy model not extensible to PHI classes and personas without forking).

Layer 6 — Foundation Models and Provider Independence

Decision. Amazon Bedrock as primary (Anthropic Claude family for complex drafting, document understanding and judging; Amazon Nova and Meta Llama for mid-tier tasks); Azure AI Foundry as secondary for every generative capability class; direct provider APIs only under BAA with zero data retention.²⁵

Why it fits. Provider independence is a resilience requirement (QA-5), a safety requirement (verifier from a different model family) and a commercial requirement (negotiating leverage). Capability-class binding makes it

25. Anthropic — Business Associate Agreements — <https://privacy.claude.com/en/articles/8114513-business-associate-agreements-baa-for-commercial-customers>

cheap.

Configuration. Every capability class has at least two eligible endpoints on different providers, each with its own eval scorecard; router policy changes are replay-tested; prompts are maintained per model family where behaviour differs, under the same version tag.

Layer 7 — Self-Hosted Inference and Optimisation

Decision. NVIDIA Triton Inference Server for all non-generative models; vLLM for generative tenant adapters; Bedrock Custom Model Import as the managed alternative for imported open-weight models.

Why it fits. The hot path runs three to five small-model inferences per message. Consolidating them on Triton with dynamic batching and TensorRT engines gives predictable single-digit to tens-of-milliseconds latency at high GPU utilisation; vLLM's PagedAttention and multi-LoRA batching let hundreds of tenant adapters share one base deployment.¹³

Configuration.

Component	Configuration
Triton — M1 encoder	TensorRT FP16 engine, dynamic batching (preferred 8/16/32, max queue delay 4ms), 2 instances per GPU on a MIG 3g slice, response cache for identical inputs
Triton — M2 sentinel	Dedicated instance group with priority scheduling, never co-batched with lower-priority models, headroom reserved at 40%
Triton — M4 ranker	FIL backend for LightGBM, CPU instances, batched per ranking call
Triton — M8 reranker	TensorRT FP16, max sequence 512, dynamic batching
vLLM — voice adapters	8B base in BF16 on L40S or H100, enable_lora with max LoRAs per batch 16, LoRA rank <= 32, prefix caching on, chunked prefill on, gpu_memory_utilization 0.90, max concurrent sequences tuned per GPU from load tests
Autoscaling	KEDA on queue depth and in-flight requests; warm minimum replicas per region; scale-to-zero only for batch pools

Managed versus self-hosted analysis. Self-hosting won where volume is high, the task is narrow and latency is tight (encoders, sentinel, ranker, voice adapters). Managed won where capability matters more than cost and volume is lower (complex drafting, document understanding, judging, RCM correspondence). The crossover was computed in the three-year TCO model including GPU reservation, engineering on-call and utilisation risk, and is re-evaluated quarterly.

Beat. TensorRT-LLM for adapters (higher peak throughput, lower adapter flexibility), TGI (less mature multi-LoRA at the time of selection), and sending all generation to hosted endpoints (roughly 4-6x higher unit cost for routine drafting at planning load).

Layer 8 — Data Platform: Operational, Streaming and Lakehouse

Decision. Aurora PostgreSQL with row-level security for operational state; MSK with Schema Registry, Debezium and the outbox pattern for events; Managed Flink for streaming; Iceberg on S3 with Glue and Lake Formation for the lakehouse; OpenLineage into Marquez for lineage.

13. Kwon et al., Efficient Memory Management for LLM Serving with PagedAttention (vLLM) — <https://arxiv.org/abs/2309.06180>

Why it fits. Scheduling, holds and intake demand relational integrity; tenant isolation must be enforced by the engine; features and training need an ordered, replayable log and a time-travel table format.

Configuration. RLS policies keyed on a session variable set from the verified tenant claim; Aurora Global Database for DR; Kafka topics partitioned by `tenant_id`; schema compatibility mode BACKWARD enforced in CI; Iceberg tables partitioned by tenant bucket and event date; Lake Formation column-level grants on PHI-derived columns; Macie scanning of the raw zone.

Beat. DynamoDB as primary operational store (weak for relational scheduling invariants), Kinesis (weaker replay and ecosystem), and third-party lakehouses (a second PHI processor to govern).

Layer 9 — Feature Platform

Decision. Feast registry and SDK, DynamoDB global tables plus Redis as online store, Iceberg as offline store, Flink and Spark as compute, SageMaker Feature Store evaluated per cell.^{5,6}

Why it fits. The ranker and autonomy policy are only defensible if the features used at decision time are reproducible at training time; Feast's point-in-time retrieval and shared on-demand transforms guarantee that, and its definitions are portable.

Configuration. Entity keys `tenant_id#entity_id`; freshness SLOs per view; PHI class on every feature enforced at materialisation; served-feature logging for skew detection.

Layer 10 — Retrieval and Knowledge

Decision. Hybrid retrieval (OpenSearch BM25 plus pgvector or OpenSearch k-NN) with cross-encoder reranking; Neo4j knowledge graph for GraphRAG and multi-hop retrieval.

Why it fits. Payer names, plan codes and modality acronyms need lexical precision; paraphrase needs dense retrieval; eligibility and referral questions are graph paths. Using each where it is structurally right avoids the most common RAG failure — forcing every question through one retrieval mode.

Configuration. Tenant pre-filtering on every query; structure-aware chunking; version-stamped embeddings with alias swaps; parameterised Cypher templates only; k-hop bounds and token budgets on multi-hop retrieval.

Beat. A separate SaaS vector database per tenant (overhead and governance cost) and Neptune (evaluated; Neo4j won on Cypher tooling, graph data science algorithms for community detection and team familiarity).

Layer 11 — Model Factory

Decision. SageMaker Training and Ray Train for compute; PyTorch, Hugging Face Transformers, PEFT, TRL and LightGBM for training; SageMaker Pipelines and Temporal for orchestration; MLflow for tracking and registry; Label Studio for labelling; Presidio plus clinical NER for de-identification; cosign for signing.

Why it fits. The eight model families need managed spot training for routine jobs, elastic distributed compute for hyperparameter search and embedding fine-tunes, and a single registry that covers classic ML and GenAI evaluation.

5. Feast — Feature view concepts — <https://docs.feast.dev/getting-started/concepts/feature-view>

6. Amazon SageMaker Feature Store — <https://docs.aws.amazon.com/sagemaker/latest/dg/feature-store.html>

Layer 12 — Trust, Safety and Red Teaming Tooling

Decision. PyRIT for automated adversarial campaigns, Giskard for vulnerability scanning and RAG evaluation, garak for probe coverage, promptfoo for prompt regression in CI, Bedrock Guardrails and Azure AI Content Safety as managed layers, NeMo Guardrails and custom validators for platform-specific rails.

Why it fits. No single tool covers indirect injection through referral documents, PHI egress, clinical-boundary violations and multilingual jailbreaks. The tooling is commodity; the attack corpora and scorers are domain-specific and owned in-house.

Layer 13 — Observability, SRE and Resilience

Decision. OpenTelemetry end to end; Grafana (Tempo, Loki, Mimir) and CloudWatch on AWS; Azure Monitor and Application Insights for the Azure path; Langfuse for LLM trace exploration; AWS Fault Injection Service and Chaos Mesh for chaos; Temporal for continuity runbooks.

Why it fits. A request that fails over from Bedrock to Azure AI Foundry must remain one trace with one cost record. Open standards make that possible across two clouds.

Layer 14 — Security and Identity

Decision. AWS KMS with per-tenant data keys and multi-region keys, Secrets Manager, IAM Identity Center, SPIFFE/SPIRE workload identity federated into Istio, Cognito or Auth0 for staff SSO with MFA and SCIM, GuardDuty, Security Hub and Macie.

Why it fits. Crypto-shredding per tenant is the only deletion mechanism that survives backups and replicas; workload identity makes every service call attributable in the audit ledger.

Layer 15 — Delivery and Infrastructure as Code

Decision. Terraform for both clouds, GitHub Actions for CI with OIDC federation, Argo CD for GitOps, Argo Rollouts for progressive delivery, ADRs in the repository.

Why it fits. Prompts, policies, graphs, adapters and models are deployable artifacts with the same promotion rules as code. One CI system across both clouds keeps the gate logic in one place.

Configuration. Pipelines run unit, contract, evaluation, red-team smoke and cost-regression stages; model and prompt releases require an eval scorecard reference; Argo Rollouts analysis templates query quality and latency SLOs during canaries and roll back automatically.

16.4 Multi-Cloud Equivalence

Capability	AWS (primary)	Azure (secondary scope)	Google Cloud (reference)
Managed foundation models	Amazon Bedrock	Azure AI Foundry	Vertex AI
Managed guardrails	Bedrock Guardrails	Azure AI Content Safety	Vertex AI safety filters
Kubernetes	EKS	AKS (not adopted)	GKE
Event log	MSK	Event Hubs (not adopted)	Managed Kafka

Capability	AWS (primary)	Azure (secondary scope)	Google Cloud (reference)
Observability	CloudWatch, Grafana	Azure Monitor, Application Insights	Cloud Monitoring
Relational + vector	Aurora PostgreSQL + pgvector	Azure Database for PostgreSQL + pgvector	AlloyDB + pgvector
Document / key-value	DynamoDB	Cosmos DB (not adopted)	Firestore / Bigtable
Training	SageMaker, Ray on EKS	Azure ML	Vertex AI Training
CI/CD	GitHub Actions	Azure DevOps (not adopted)	Cloud Build

16.5 Deliberately Not Selected

Technology pattern	Reason
Managed agent platforms as the orchestration core	Opaque planning loops and limited control over typed state, budgets and trajectory evaluation
Conversational multi-agent frameworks for production flows	Natural-language inter-agent messaging produces non-deterministic trajectories and unbounded cost
A single frontier model for everything	Cost, latency, correlated verifier failure and vendor concentration
Full fine-tuning of large models per tenant	Adapters deliver voice fidelity at a fraction of the cost with deletable, tenant-scoped artifacts ¹⁰
RLHF with a learned reward model for voice	DPO on edit pairs meets the objective with less instability ¹¹
Symmetric active-active multi-cloud data plane	Doubles PHI compliance surface for a risk the model did not justify

10. Hu et al., LoRA: Low-Rank Adaptation of Large Language Models — <https://arxiv.org/abs/2106.09685>

11. Rafailov et al., Direct Preference Optimization — <https://arxiv.org/abs/2305.18290>

17 Data Architecture and Interoperability

17.1 Operational Stores

Store	Contents	Pattern
Aurora PostgreSQL	Inquiries, threads, prospects, clinicians, slots, holds, appointments, intake packets, referrals, tenant configuration	Event-sourced aggregates with outbox; RLS by tenant
Encrypted object storage	Message bodies, recordings, transcripts, documents	Per-tenant prefixes and keys; lifecycle policies aligned to retention
Audit ledger	PHI access, model calls, tool calls, human decisions	Append-only, hash-chained, separate account
Event log	Domain events, AI events (proposals, verdicts, decisions), outcome events	Tenant-partitioned, schema-registered, 30-day hot retention, tiered to lake

17.2 Canonical AI Decision Record

Every consequential AI proposal is persisted as a decision record, the join key between the product, the audit ledger and the training pipeline.

```
AIDecisionRecord
  decision_id, tenant_id, inquiry_id, workflow_run_id, trace_id
  decision_type:      DRAFT | MATCH_RANKING | SLOT_PROPOSAL | ESCALATION | AUTONOMY_MODE
  inputs_manifest:    { context_slots[], feature_vector_ref, kb_chunk_refs[], data_classes[] }
  artifact_versions:  { agent, prompt, policy_bundle, graph, router_policy, model_endpoint, adapter, ranker }
  output_ref, output_hash, calibrated_confidence, risk_class
  verifier:           { status, defects[], model_endpoint }
  execution:           { mode, executed_by: SYSTEM | user_id, executed_at }
  human_feedback:     { action: APPROVED | EDITED | REJECTED | OVERRIDDEN, edit_ref, reason_code }
  outcomes:           { replied_at, booked_at, showed, retained_90d } # appended as labels mature
```

17.3 Healthcare Interoperability

The internal model is FHIR R4-shaped — Patient-equivalent prospect records, Practitioner, PractitionerRole, Schedule, Slot, Appointment, QuestionnaireResponse and Consent — so that EHR connectors map to a stable canonical form and agents never see vendor-specific payloads.²³ EHR synchronisation for SimplePractice and TherapyNotes is bidirectional for scheduling and handoff, consistent with the product commitment; clinical documentation remains in the EHR. Connectors implement reconciliation: nightly comparison of platform appointments with EHR state, with discrepancies raised as operational tasks rather than silently overwritten.

23. HL7 FHIR R4 — <https://hl7.org/fhir/R4/>

18 Canonical APIs

Surface	Endpoint family	Contract notes
AI gateway	POST /v1/infer, POST /v1/embed, POST /v1/transcribe, POST /v1/rerank	Mandatory tenant_id, intent, data_class, budget, trace_context; typed response with usage and cost
Agents	POST /v1/agents/{agent}/invoke, GET /v1/runs/{run_id}, POST /v1/runs/{run_id}/resume	Invocations are asynchronous; resume carries a human decision payload
Review	GET /v1/reviews?queue=, POST /v1/reviews/{id}/decision	Decision schema doubles as training label schema
Features	POST /v1/features/online:get, POST /v1/features/historical:materialize	Online batch gets by entity keys; historical materialisation by entity dataframe and as-of time
Models	POST /v1/models/{id}/promote, POST /v1/models/{id}/rollback	Promotion requires scorecard reference and approver identity
Governance	GET /v1/ai-inventory, POST /v1/killswitch	Kill switch by scope: fleet, tenant, intent, endpoint

19 Trade-Offs and Decisions I Owned

As sole Principal AI Architect I owned the decisions below end to end: framing the options, commissioning the evidence, making the call, recording it as an ADR and carrying the consequences. Each entry states the tension, the evidence that settled it and the cost I knowingly accepted.

19.1 The Decision Register

ADR	Decision I made	Options on the table	Evidence that settled it	Cost I accepted
ADR-01	Deterministic services own all business state; models only propose	Agents with direct write access to scheduling and messaging	Failure-mode analysis of QA-6: a single injected instruction could double-book or message the wrong person	More code at the boundary; slower feature velocity in the first two quarters
ADR-02	Temporal for lifecycle, LangGraph for reasoning, graphs executed as activities	One agent framework for both; Step Functions plus agents	Spike: a 4-day follow-up cadence with human takeover, deploys mid-flight and a provider outage; only the dual-runtime design passed without duplicate sends	Two runtimes to operate and staff
ADR-03	Supervisor hierarchy with typed handoffs; no free-form inter-agent chat	Group-chat multi-agent patterns	Trace replay: free-form topology produced 3.1x token spend and non-reproducible tool sequences on the same inputs	Less emergent flexibility
ADR-04	Safety sentinel as a fine-tuned ensemble, fail-closed	Rely on LLM refusal behaviour and provider moderation	Benchmark on the adjudicated set: LLM-only detection missed indirect Spanish-language ideation below the recall floor; the ensemble met ≥ 0.99	Clinical labelling cost; ongoing per-language maintenance
ADR-05	Self-host high-volume narrow models (Triton, vLLM); buy frontier capability	All-managed; all-self-hosted	Three-year TCO plus p95 benchmarks per capability class	GPU capacity planning and on-call for serving
ADR-06	Per-tenant LoRA adapters with DPO on edit pairs	Prompt-only style; full fine-tuning; shared voice model	Blind staff preference test and edit-distance on held-out tenants; deletability requirement from QA-7	Adapter lifecycle management at hundreds of tenants
ADR-07	Feature platform with point-in-time correctness before the ranker ships	Ad-hoc SQL features; ship the ranker first	Offline-online gap analysis: ad-hoc features overstated offline NDCG materially relative to shadow traffic	One quarter of platform investment before visible ranking gains
ADR-08	Graded outcome labels (booked, attended, retained) for ranking	Optimise for booking	Booking-optimised ranker increased bookings but lowered 90-day retention in shadow analysis	Delayed labels and slower iteration
ADR-09	Placeholder rehydration keeps identifiers out of prompts	Instruction-based PHI avoidance	Red-team campaigns extracted identifiers from instruction-only designs; placeholders made extraction structurally impossible	A template engine in the dispatch path
ADR-10	Verifier from a different model family than the drafter	Self-critique by the same model	Correlated-error analysis: same-family judging approved a large share of its own factual errors	An additional provider dependency
ADR-11	Autonomy as a policy with a live quality-feedback term	Static per-tenant autonomy configuration	Simulation of tenant drift: static autonomy kept auto-sending through a quality regression for days	Occasional automatic demotions that product had to explain to customers

ADR	Decision I made	Options on the table	Evidence that settled it	Cost I accepted
ADR-12	Cross-region DR for state, cross-cloud failover for inference only	Symmetric active-active multi-cloud	Risk model and TCO: full multi-cloud data plane roughly doubled compliance surface for a scenario outside the risk appetite	Dependence on AWS for state-plane recovery
ADR-13	Neo4j for the knowledge graph, strictly derived from the operational store	Neptune; graph as system of record	Spike on eligibility-path and community-detection queries; drift analysis of authoritative-graph designs	A CDC pipeline and a second query language
ADR-14	Semantic caching only for non-PHI classes	Semantic cache everywhere for cost	Leak test: near-duplicate prompts across tenants produced cross-tenant cache hits in a naive design	Lower cache hit rate on the client-facing path
ADR-15	Istio egress gateway with destination allow-lists	Security groups and application-level checks only	Exfiltration path analysis identified agent-reachable HTTP paths not covered by application checks	Mesh operational overhead
ADR-16	Low-resource languages capped at human-review autonomy	Uniform autonomy across languages	Per-language recall and translation adequacy below floor for the long tail	Slower replies in those languages

19.2 The Three Hardest Calls

Latency versus safety on the first reply. Product wanted the generative reply inside 30 seconds with safety checks running asynchronously. I refused to take the safety verdict off the critical path. Instead I split the reply into a guaranteed acknowledgement path and an enriched follow-on, moved the sentinel's first two layers onto Triton to bring its p95 to 150ms, and parallelised language and extraction around it. We met the reply objective with the safety barrier intact; the concession was a slightly less personalised first message on the slowest percentile.

Managed versus self-hosted inference. Engineering leadership was split: one camp wanted all-managed for operational simplicity, another wanted self-hosting everywhere for cost. I framed it per capability class, commissioned a two-week spike measuring p95, p99 and cost per 1,000 calls for each class on both options, and built the three-year TCO including GPU reservations and on-call cost. The result — self-host the narrow, high-volume models, buy frontier capability — was accepted because it was arithmetic, not opinion.

Autonomy that can be taken away. Customers value automation, and a system that demotes itself looks less capable. I owned the decision that autonomy is earned per tenant and per intent and is automatically withdrawn when the rejection rate breaches its control limit. It cost us some uncomfortable customer conversations; it also meant the platform never auto-sent through a quality regression.

20 Trust, Safety and Cost Governance

20.1 Governance Model

Trust, safety and cost are governed as one system because they fail together: a jailbreak is a safety incident, a data-exposure incident and usually a cost anomaly at the same time. The governance model has four control layers — preventive, detective, responsive and assurance — applied to three risk domains.

Layer	Safety	Security and privacy	Cost
Preventive	Clinical boundary rails, sentinel, autonomy policy, low-resource language caps	PHI minimisation, placeholders, capability tokens, egress allow-lists, tenant-keyed caches	Budgets per tenant and intent, small-model-first routing, token ceilings
Detective	Crisis-path canaries, verifier vetoes, conversation-level escalation tracking	Guardrail hits, injection classifiers, cross-tenant leakage tests, Macie, GuardDuty	Spend-velocity anomaly detection, cost-per-inquiry drift
Responsive	Kill switches at fleet, tenant, intent and endpoint; human-only mode	Capability revocation, key rotation, incident response	Automatic model downgrade, tenant throttling, budget freeze
Assurance	Adjudicated golden sets, release gates, quarterly clinical review	Red-team programme, AI-agent-led audit, penetration tests	Monthly cost governance review, three-year TCO refresh

20.2 Threat Model

The threat model follows the OWASP Top 10 for LLM applications, specialised for an agentic, PHI-adjacent, multi-tenant system.²⁶

Threat	Platform-specific vector	Primary controls
Prompt injection, direct	Client messages instructing the Assistant to ignore policy, reveal other clients or promise outcomes	Instruction hierarchy, input classifiers, persona-scoped tools, verifier veto
Prompt injection, indirect	Referral PDFs, directory text, email signatures and calendar event descriptions carrying instructions	Untrusted-content isolation in delimited blocks, no tool access from document-reading nodes, capability tokens bound to the current inquiry
Sensitive information disclosure	Cross-tenant leakage through caches, retrieval or adapters; PHI in logs	Tenant-keyed caches, pre-filtered retrieval, tenant-confined adapters, encrypted PHI-tagged telemetry
Excessive agency	Agent sends, confirms or cancels without warrant	Autonomy policy external to agents; dispatch only from workflow activities; human tokens for irreversible actions
Improper output handling	Model output parsed as commands or rendered as markup	Typed outputs only; output encoding; no model output reaches a shell, query or template engine unescaped
Data and model poisoning	Malicious or low-quality edits entering preference data	Edit-pair filters, contributor reputation, batch anomaly detection, gated snapshot promotion
Supply chain	Compromised weights, adapters or packages	Signed artifacts, pinned digests, provenance checks, SBOMs
System prompt leakage	Extraction of policy prompts	No secrets in prompts; prompts treated as non-confidential
Unbounded consumption	Loops, token floods, denial-of-wallet	Runtime budgets, per-tenant quotas, spend-velocity alerts

26. OWASP Top 10 for LLM Applications — <https://genai.owasp.org/llm-top-10/>

Threat	Platform-specific vector	Primary controls
Vector and embedding weaknesses	Poisoned KB content, embedding inversion	Approval-gated publication, tenant-partitioned indexes, no raw PHI in embedded text

20.3 Red Teaming Programme

Red teaming is a standing programme with its own backlog, not a pre-launch event.

Track	Tooling	Cadence	Scope
Automated adversarial campaigns	PyRIT orchestrators (multi-turn crescendo, prompt-converter chains for encoding, translation and obfuscation) with rubric and classifier scorers	Nightly against staging; on every graph, prompt or model release	Every client-facing agent and persona
Vulnerability scanning	Giskard scans for injection, harmful content, hallucination, sensitive-information disclosure and stereotypes; RAG evaluation test-set generation	Weekly and per release	Drafting, retrieval-grounded answers, staff copilots
Probe coverage	garak probe suites (encoding, DAN-style jailbreaks, leakage, malware-generation refusal)	Per model or endpoint onboarding	Every new endpoint in the catalog
Prompt regression	promptfoo assertions in CI	Every pull request touching prompts or policies	Tone, policy, refusal and format invariants
Domain attack corpora	In-house, clinically reviewed: indirect crisis phrasing, bilingual code-switching, minors, coercion, false urgency	Continuously extended from incidents and near-misses	Safety sentinel, clinical boundary rail
Human red team	Security engineers and clinical reviewers	Quarterly, plus before major releases	End-to-end adversarial scenarios across channels

Attack success rate (ASR) is tracked per attack family, per persona and per language. The release gate is zero successful attacks on the critical families — crisis mishandling, PHI disclosure, cross-tenant access, unauthorised side effects — and no regression beyond tolerance on the others.

20.4 Adversarial Prompt Injection Defence in Depth

Layer	Control	Failure it catches
1. Channel	Normalisation, Unicode confusable folding, hidden-text stripping from HTML and PDFs, OCR of images in documents	Invisible instructions, homoglyph smuggling
2. Classification	Injection classifier on all untrusted content; Bedrock Guardrails prompt-attack filter on Bedrock routes	Known and paraphrased injection patterns
3. Context construction	Untrusted content in labelled, delimited blocks; instruction hierarchy in system policy; spotlighting of data versus instructions	Instructions blending into context
4. Architecture	Document-reading and tool-calling separated into different nodes; the node that reads a referral PDF has no tools	Indirect injection leading to action
5. Authorisation	Capability tokens bound to the current inquiry and verb; persona-scoped tool manifests	Privilege escalation after a successful injection
6. Egress	Recipient allow-list for outbound messages; Istio egress gateway allow-list; PHI egress rail	Exfiltration to attacker-controlled destinations

Layer	Control	Failure it catches
7. Verification	Different-family verifier checks every outbound artifact for policy conflicts and unsupported claims	Subtle manipulations that pass earlier layers
8. Detection and response	Injection attempts logged as security events; tenant-level alerting; automatic autonomy demotion	Campaigns against a tenant

20.5 Data Exfiltration Path Analysis

Every path from a PHI source to an external sink was enumerated, assigned a control and covered by an automated test. New integrations cannot ship until their paths are added.

Source	Sink	Path	Control	Test
Message store	SMS or email to a non-client	Agent draft addressed to an injected recipient	Recipient must equal the inquiry's verified contact; dispatch is deterministic	Injection corpus with alternate recipients
Operational DB	External HTTP endpoint	Agent tool invoking an arbitrary URL	No generic HTTP tool exists; mesh egress allow-list	Chaos test attempting egress from every agent pod
Knowledge base	Another tenant	Retrieval without tenant filter	Pre-filter enforced in the query layer and RLS	Cross-tenant retrieval tests per release
Prompt context	Model provider logs	PHI in prompts to a non-BAA endpoint	Gateway data-class eligibility; zero-retention endpoints only	Policy unit tests; synthetic PHI canaries
Traces and logs	Observability vendors	PHI in span attributes	PHI-tagged attributes encrypted; redaction processors in the OTel collector	Log scanning with Macie-style detectors
Training data	Shared model weights	Memorisation of identifiers	De-identification recall gate; tenant adapters isolated	Membership-inference and canary-extraction tests
Semantic cache	Another tenant	Near-duplicate prompt hit	Tenant and data class in key; disabled for PHI classes	Cross-tenant cache probe

20.6 Tenant-Level Network Policy Enforcement

Each cell namespace starts from default-deny ingress and egress Kubernetes NetworkPolicies. Istio AuthorizationPolicies allow only declared service identities to call declared methods, and a tenant-dedicated cell's workloads can only reach that cell's data stores. All outbound traffic leaves through the egress gateway, where destinations are allow-listed per namespace. The policies are generated from the service catalogue and verified in CI by policy tests and in production by continuous connectivity probes that must fail.

20.7 AI-Agent-Led Compliance Audit

Compliance evidence is collected continuously by a read-only Compliance Agent, not assembled by hand before an audit.

Element	Design
Controls catalogue	HIPAA Security Rule safeguards, BAA obligations, internal AI policy and NIST AI RMF functions mapped to testable control statements ^{27,28}
Evidence sources	CloudTrail, Config, Security Hub, IAM Access Analyzer, Kubernetes policy reports, audit ledger, MLflow registry, red-team results, access reviews

27. HHS — Summary of the HIPAA Security Rule — <https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html>

28. NIST AI 600-1 — Generative AI Profile of the AI RMF — <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>

Element	Design
Agent permissions	Read-only, metadata-level; no PHI access; runs under its own persona profile
Output	Evidence units — a control ID, the query that proved it, the result, a timestamp and a hash — written to the audit account
Posture improvement	Failed controls open tickets with owners and SLAs; trend of control pass rate reported monthly
Human role	Compliance lead reviews and attests; the agent never attests

An evidence unit is also the unit of cost for compliance: cost per evidence unit (compute, queries and model tokens per verified control check) is tracked so that continuous audit remains cheaper than periodic manual audit.

20.8 Clinical Boundary and HIPAA Safeguards

The platform is operational, not clinical. Agents do not diagnose, recommend treatment, interpret screener scores for the client or counsel. PHQ-9 and GAD-7 scores are computed deterministically and delivered to the clinician; agents may confirm completion but never discuss results. Any crisis signal suppresses generative outbound, sends a pre-approved message with crisis resources including 988 for U.S. contacts, and pages the tenant's on-call.¹⁴

Safeguard	Implementation ²⁷
Access control	Tenant-scoped RBAC plus ABAC on data class, persona and purpose; break-glass with justification and alerting
Audit controls	Append-only, hash-chained audit ledger for every PHI access, model call with PHI class, tool call and human decision
Integrity	Event sourcing with checksums; signed model artifacts; immutable dataset snapshots
Authentication	SSO with MFA for staff; SPIFFE workload identities for services
Transmission security	TLS 1.2+ externally, Istio mTLS internally, private connectivity to model providers

20.9 Cost Governance

Unit Economics

The planning load implies roughly 38k input and 3k output tokens per inquiry lifecycle before optimisation. Cost is governed at four units:

Unit	Definition	Use
Cost per inquiry lifecycle	All model, retrieval, speech and orchestration cost from capture to close	Primary SLO; per tenant cohort
Cost per booked first session	Lifecycle cost divided by first sessions attended	Links AI spend to revenue
Cost per evidence unit	Compliance-agent cost per verified control check	Keeps continuous audit economical
Cost per 1,000 drafts by capability class	Normalised serving cost	Managed-versus-self-hosted and routing decisions

14. SAMHSA — 988 Suicide and Crisis Lifeline — <https://www.samhsa.gov/mental-health/988>

27. HHS — Summary of the HIPAA Security Rule — <https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html>

Optimisation Levers

- 1. **Right-size the model.** Extraction on the M1 encoder and routine drafting on tenant adapters removes the majority of frontier tokens from the hot path.
- 2. **Prefix and KV caching.** Stable system, policy and style-card prefixes first; high provider prefix-cache hit rates and KV reuse on vLLM.¹³
- 3. **Context discipline.** Typed facts and summaries replace raw history; retrieval k capped per agent; gateway-enforced token ceilings.
- 4. **Batch where latency permits.** Digest narration, directory copy, backfills and evaluation judges on batch inference.
- 5. **GPU efficiency.** MIG partitioning, dynamic batching and utilisation-driven autoscaling on Triton and vLLM pools.

Per-Tenant Attribution, Anomaly Alerting and Chargeback

Every model and tool call writes a cost-ledger record keyed by tenant, persona, intent, agent, endpoint and workflow run. Spend velocity per tenant and intent is monitored with seasonal baselines; a breach triggers automatic model downgrade for that tenant and intent before throttling, and pages FinOps above a monetary threshold. Showback by tenant, tier and capability class feeds pricing and packaging; because pricing is tied to practice scale, chargeback is internal — it identifies tenants whose usage profile erodes margin and informs tier boundaries.

Three-Year TCO Model

Every infrastructure decision above a monetary threshold required a three-year TCO covering compute and GPU reservations, managed-service fees, token spend under the planning load and growth scenarios, storage and egress, people cost for operation and on-call, and exit cost. The model is refreshed quarterly against actuals; decisions whose assumptions drift by more than 20% are re-opened.

Monthly Cost Governance Review

A monthly review, which I chaired, covered cost per inquiry by cohort, top ten tenants by margin erosion, routing mix, cache hit rates, GPU utilisation, anomalies and their resolution, and any TCO assumption drift. Actions were owned and tracked like incidents.

20.10 Release Gates I Held

Gate	Evidence required
Safety	Per-language recall at or above floor on adjudicated sets; zero critical red-team successes; kill switches exercised
Privacy	Context-manifest audits proving minimisation; de-identification recall audit; BAA coverage for every PHI route
Tenancy	Automated cross-tenant leakage tests across caches, retrieval, features, adapters and network
Quality	Node, trajectory and outcome evaluations at or above champion; calibrated confidence
Fairness	Exposure and outcome parity for ranking; language parity for drafting
Reliability	SLOs, dashboards, provider failover tested, degraded modes demonstrated

13. Kwon et al., Efficient Memory Management for LLM Serving with PagedAttention (vLLM) — <https://arxiv.org/abs/2309.06180>

Gate	Evidence required
Cost	Cost per inquiry within plan at planning load; anomaly alerts armed
Governance	AI inventory entry, model and data cards, approvals, rollback tested

Jyotirmoy Bardhan

21 Authority

21.1 Decision Rights

My authority on the programme was explicit and written into the engineering operating model, so that it did not depend on persuasion in every meeting.

Authority	Scope	How it was exercised
ADR ownership	Every architecture decision with cross-team or irreversible impact	I authored or approved all 16 platform ADRs; teams authored local ADRs within the guardrails they set
Technology selection and build-versus-buy adjudication	Every new runtime dependency, managed service or vendor	Spike with a pre-agreed metric, TCO, security and BAA review, then my decision recorded as an ADR
Managed-versus-self-hosted approval	Inference, data stores, orchestration runtimes	Approval required above a monthly-cost threshold, with the three-year TCO attached
Control and data plane design ownership	Gateway, policy engine, persona profiles, orchestration, event model, tenancy model, feature and knowledge planes	I owned the reference architecture and the interface contracts between planes
Standing veto over governance-violating proposals	Any design that bypassed the gateway, stored PHI outside approved stores, gave agents direct write access, or weakened tenant isolation	Used sparingly and always with a documented alternative
Trust and safety release gating	Every release touching client-facing AI behaviour	Joint sign-off with the clinical safety lead; I could block, and did
Evidence-lineage contract ownership	The decision record and its lineage to data, features, prompts and models	Schema changes required my approval
Budget-linked infrastructure approval	Infrastructure spend above thresholds	Tiered thresholds: team lead, then me, then CTO

21.2 Exercising the Veto

I used the veto four times in eighteen months. Each time I brought an alternative that met the proposing team's goal.

Proposal	Why I blocked it	What we did instead
Let the drafting agent call the SMS API directly to cut latency	Violated ADR-01; an injected recipient would become a real send	Dispatch from a workflow activity after the autonomy decision; latency cost under one second
Enable a global semantic cache for all prompts to reduce spend	Cross-tenant PHI leakage risk (ADR-14)	Tenant-keyed semantic cache for non-PHI classes plus prefix caching
Ship a new frontier model as the default drafter based on vendor benchmarks	No platform eval scorecard; verifier from the same family	Two-week evaluation on platform golden sets; adopted for complex threads only
Store call recordings in a transcription vendor's bucket for convenience	Processor outside BAA coverage and retention controls	Recordings stay in the tenant-keyed store; vendor processes via streaming with zero retention

21.3 Authority Boundaries

Authority was bounded as deliberately as it was granted. Product owned what to build and in what order; the clinical safety lead owned the safety taxonomy and clinical boundary; the CISO owned the security programme; the CTO owned budget above my thresholds. My role was to own how the platform was built and to guarantee that its trust, safety, cost and resilience properties held.

22 Leadership

22.1 Team Structure

I led a multidisciplinary organisation of 28 engineers at peak across seven teams, without direct line management of most of them — leadership was through architecture, evidence and clarity of ownership.

Team	Size at peak	Owns	Interfaces
AI Platform	5	Gateway, router, context compiler, guardrails, prompt and policy registry, persona profiles	All agent and product teams
Agent Engineering	5	Supervisor, specialist agents, graph definitions, trajectory evaluation	Product squads; clinical safety
ML Platform	4	Feature platform, training pipeline, registry, Triton and vLLM serving	Data platform; applied ML
Applied ML	4	M1-M8 models, experiments, fairness analysis	Product analytics; clinical safety
Data Platform	4	Event log, lakehouse, CDC, lineage, semantic metrics layer	All teams
SRE and DevOps	4	SLOs, mesh, IaC, CI/CD, chaos, DR, cost tooling	All platform teams
QA and Evaluation	2	Golden sets, red-team automation, release evidence	Clinical safety; every release

22.2 How I Led

Sequencing cross-team dependencies under compressed timelines. The critical path ran through the gateway, the event model and the tenancy model; every other team depended on them. I front-loaded those three in the first quarter, published their contracts before their implementations, and let downstream teams build against stubs and contract tests. That kept seven teams moving in parallel and moved first production tenants to month 7.

Architecture review facilitation. A weekly architecture review with a fixed agenda: new ADR proposals, contested trade-offs, and one deep dive on a production trace. Proposals came with a one-page problem statement, options, evidence and a recommendation; decisions were made in the room or assigned a spike with a deadline.

Prototype-spike-driven decision validation. Contested decisions were never settled by seniority. Each became a two-week spike with a pre-agreed metric, a named owner and a decision date. Eleven of the sixteen platform ADRs were settled this way.

Evidence-based debate resolution. The managed-versus-self-hosted debate, the sentinel design and the vector-store topology were settled by recall and latency benchmarks, not preference. When the data contradicted

my own initial position — as it did on the vector store, where I had initially favoured a dedicated vector database for all tenants — I changed the decision and said so publicly in the review.

Engineering leadership alignment on contested trade-offs. For the two decisions with the largest organisational impact — the dual-runtime orchestration and cross-cloud inference failover — I took the evidence to the CTO and engineering directors before the architecture review, so that the review ratified an aligned position rather than surfacing a conflict in front of the teams.

Raising the bar. I introduced trace replay as a release requirement for graph changes, cost regression as a CI stage, and red-team smoke tests in every pipeline — each initially resisted as slowing delivery, each later credited by the teams with preventing a production incident.

22.3 Programme Leadership

Phase	Months	What I drove
Foundation	1-4	Reference architecture, tenancy and event models, gateway, audit ledger, sentinel L1-L2, first ADRs
First production tenants	5-7	Lifecycle workflows, acknowledgement path, human review, voice pipeline, first cell in production
Intelligence	8-12	Feature platform, ranker, V1 voice adapters, verifier diversity, GraphRAG, Triton consolidation
Guarded autonomy and resilience	13-16	Autonomy policy with quality feedback, Azure inference failover, continuity runbook, chaos programme
Learning platform	17-18	DPO adapters, forecasting, compliance agent, dedicated cells for the largest networks

The resilience and continuity programme and the simulation-based validation governance were mine specifically: I set the RTO/RPO tiers, chaired the quarterly continuity review, and signed off each game-day result.

23 Business Outcome

23.1 Outcomes Across Pilot Practices

The platform's purpose was measurable practice outcomes, and they were measured through the same event-sourced funnel the platform runs on. Figures are aggregate across pilot practices; individual practice results vary.

Outcome	Before	After
Average response time to new inquiries	9h 12m	2h 14m (68% faster)
Median first reply	Baseline	1m 42s
Same-day reply rate	Baseline	94%
Inquiry-to-session conversion	Baseline	+38%
Right-fit clients matched	First-available matching	2.4x versus first-available
Time to first session	Baseline	6 days sooner
In-network match rate	Baseline	+41%
90-day client retention	Baseline	+27%
Administrative time	Baseline	12 hours saved per clinician per week
Booked sessions per month after 12 months	Baseline	+162% average lift
Inquiries lost in inbox	Untracked	0
Onboarding	—	94% of clients onboarded in under 30 days

23.2 How the Architecture Produced the Outcomes

Outcome	Architectural cause
Response time and same-day reply	Durable acknowledgement path, after-hours autonomy for approved intents, 27s p95 compute budget with retry headroom
Conversion	Faster first contact, localised full-thread communication, nurture cadence driven by engagement models
Right-fit matching and retention	Graded outcome labels, point-in-time features, propensity-corrected learning from overrides, calibrated scores
In-network rate and time to first session	Panel eligibility as hard constraint, live capacity features, transactional holds
Staff time saved	Practice-voice drafting with high acceptance, automated tagging, voicemail triage, grounded digest
Zero lost inquiries	Idempotent capture, cross-channel entity resolution, event-sourced lifecycle

23.3 Platform Economics

Measure	Result
Hot-path model mix	~70% of invocations served by encoders, rankers and tenant adapters rather than frontier models
Billable input tokens	45-60% reduction from prefix caching and context discipline
Cost per inquiry lifecycle	Held within plan through fleet growth; attributable per tenant
Compliance	Continuous evidence for the controls catalogue; customer security reviews answered from the evidence store
Resilience	Provider brown-outs absorbed by cross-cloud routing without customer-visible latency breaches

24 Architectural Impact

24.1 What the Architecture Changed

It made safety structural. The sentinel as a hard barrier, placeholders instead of identifiers, agents without direct write access, capability tokens and mesh-level egress control mean the most important safety and privacy properties hold even when a model misbehaves. They do not depend on a prompt.

It turned human oversight into a learning signal. The review schema, the decision record and outcome events were designed together, so every approval, edit, override and outcome is a labelled training row. Staff edits train voice adapters through DPO; overrides train the ranker through propensity-corrected learning; rejections tune autonomy. The loop compounds per tenant.

It made the platform provider-independent in practice, not in slides. Capability-class binding, MCP tools and a policy-driven router meant that switching providers for a capability class became a replay-tested configuration change, proved under live brown-outs.

It made cost a first-class engineering property. Cost per inquiry as an SLO, per-tenant attribution, small-model-first routing on Triton and vLLM, and monthly governance kept unit economics viable under scale-based pricing.

It made resilience measurable. RTO and RPO per capability tier, continuity runbooks as workflows and scheduled chaos experiments turned recovery objectives into numbers from the last run.

24.2 Reusable Patterns Established

Pattern	Reuse
Dual-runtime orchestration (durable workflow plus agent graph)	Adopted as the organisation's standard for any long-lived AI workflow
Capability-class model gateway with PHI-class routing	Shared service for all AI features across products
Persona profiles as signed policy bundles	Template for new personas and partner integrations
Decision record with evidence lineage	Basis for audit, training and customer transparency
Autonomy with live quality feedback	Standard control for any automated outbound action
Exfiltration path analysis as a release artifact	Required for every new integration

Pattern	Reuse
Three-year TCO as an approval input	Standard for infrastructure decisions above threshold

24.3 Principal Assessment

The defensible position of this platform is not the language model; every competitor can call the same models. The durable advantage is the closed loop: a decision is made with reproducible features, a human edits or overrides it, the outcome arrives days later, the three are joined into a labelled record, and the next model — ranker, adapter, sentinel, autonomy policy — is measurably better for that tenant. The feature platform makes the loop reproducible, the orchestration fabric makes it safe and observable, the training pipeline makes it compound, and the governance model makes it trustworthy. None of the platform's hardest risks — low-resource-language safety, feedback-loop bias, tenant memorisation, cost exposure, provider outages — was solved by a better prompt. Each was solved by an architectural boundary, a measurement and a control loop. That is the difference between a GenAI feature and an AI-native platform.